



BASE24[®]

IEP Objects Reference Manual

© 2000 by ACI Worldwide Inc. All rights reserved.

All information contained in this document is confidential and proprietary to ACI Worldwide Inc. This material is a trade secret and its confidentiality is strictly maintained. Use of any copyright notice does not imply unrestricted or public access to these materials. No part of this document may be photocopied, electronically transferred, modified, or reproduced in any manner without the prior written consent of ACI Worldwide Inc.

ACI, BASE24, BASE24-atm, BASE24-pos, BASE24-telebanking, BASE24-teller, and NET24-XPNET are trademarks or registered trademarks of ACI Worldwide Inc., Transaction Systems Architects, Inc., or their subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.

Print Record

07/2000, Version 3 (Reissue)

03/97, Version 2 (Reissue)

08/95, Version 1 (New)

Contents

Preface	ix
Conventions Used in This Manual	xiii
Section 1	
Introduction	1-1
Integrated Endpoint Process	1-2
Assumptions	1-3
Supported Message Formats	1-3
Transport Layers	1-3
Source Class	1-3
Message-Based Routing	1-4
Routing Flexibility	1-4
Usage	1-4
Source Class	1-5
Message Class	1-5
Destination File (DEST)	1-5
Routing Methods	1-6
Symbolic Destination Routing	1-7
Context Management	1-8
Measure Counter Manipulation	1-9
Network Overload Management	1-10
Integrated Endpoint Process Objects	1-11
Kernel Objects	1-11
Integrated Endpoint Objects	1-11
Core Objects	1-12
Building Processes	1-12

Introduction *continued*

Integrated Endpoint Extended Memory Tables	1-13
Context Extended Memory Table.....	1-13
Destination File Extended Memory Table	1-13
Integrated Endpoint Processing Overview	1-14
External Messages	1-14
Internal Integrated Server Process Messages	1-15
Procedural BASE24 Messages	1-15
Transaction Paths	1-16
Integrated Endpoint Process Member Functions	1-17
All Lexicons	1-18
Context Manager to Object Lexicon.....	1-19
Endpoint Class Handler to Endpoint Class Handler Services Lexicon	1-20
Endpoint Class Handler Services to Object Lexicon	1-22
Message ISO to Object Lexicon.....	1-25
Object to Context Manager Lexicon.....	1-27
Object to Destination Lexicon	1-28
Object to Endpoint Class Handler Services Lexicon	1-29
Object to Message ISO Lexicon.....	1-30
Destination File Documentation	1-31

Section 2

Context Manager Object.....	2-1
Overview.....	2-2
Context Storage.....	2-2
Context Retrieval	2-3
Context Processing	2-3
Normal Processing	2-5
Context Extended Memory Table.....	2-5
Context Storage and Retrieval	2-5
Context Storage.....	2-6
Context Retrieval	2-6
Timeout Processing.....	2-7
STATUS Command	2-7

Context Manager *continued*

Configuration	2-8
Kernel Configuration	2-8
LCONF Assigns and Params	2-8
Member Functions	2-9
Member Functions Sent	2-9
Member Functions Received	2-9

Section 3

Destination Object	3-1
Overview	3-2
Normal Processing	3-4
Initialization	3-4
Destination Retrieval	3-4
Wildcard Routing	3-4
Search Key	3-5
Search Processing	3-9
Configuration	3-11
Kernel Configuration	3-11
LCONF Assigns and Params	3-11
Member Functions	3-12
Member Functions Sent	3-12
Member Functions Received	3-12

Section 4

Endpoint Class Handler Object	4-1
Overview	4-3
Initialization	4-6
Normal Processing	4-7
Message Reception	4-7
Message Conversion	4-7
Fixed Structure Messages	4-7
ISO 8583 Bit-Mapped Messages	4-8
Field Separated Messages	4-8
Message Routing	4-8
Message Failure	4-9

Normal Processing *continued*

Device-Specific Functions	4-9
Network Overload Management	4-10
Optional Processing	4-11
EMT Notify Member Functions	4-11
Exception Handling	4-11
Extended Memory Table Usage	4-11
Generic Endpoint Class Handler Object	4-12
Configuration	4-13
Kernel Configuration	4-13
LCONF Assigns and Params	4-13
Member Functions	4-14
Member Functions Sent	4-14
Member Functions Received	4-15

Section 5

Endpoint Class Handler Services Object	5-1
Overview	5-2
Normal Processing	5-4
External Message Routing	5-4
Endpoint Class Handler Object Messages	5-4
Core Object Messages	5-4
Destination Retrieval	5-5
Context Storage and Retrieval	5-5
External Message Formatting	5-6
Timed Out Message Processing	5-6
External Message Sending	5-6
Measure Counter Manipulation	5-7
Message Variant	5-9
Message Type	5-9
Position Counter	5-10
Message Routing Timer Processing	5-12
Configuration	5-13
Kernel Configuration	5-13
LCONF Assigns and Params	5-13

Endpoint Class Handler Services Object *continued*

Member Functions	5-14
Member Functions Sent	5-14
Member Functions Received	5-15

Section 6

Message ISO Object.	6-1
Overview	6-2
Normal Processing	6-4
ISO External Message Format	6-4
Message Compression	6-5
Message Expansion	6-5
Configuration	6-7
Kernel Configuration	6-7
LCONF Assigns and Params	6-7
Member Functions	6-8
Member Functions Sent	6-8
Member Functions Received	6-9

Section 7

Integrated Endpoint Process Transaction Processing	7-1
Transaction Request Authorized by an Integrated Authorization Server Process	7-2
Transaction Response Generated by an Integrated Authorization Server Process	7-7
Financial Completion Authorized by an Integrated Authorization Server Process	7-11
Financial Transaction Authorized by an Endpoint	7-15
Context Timeout	7-19
Network Overload Detected	7-24

Glossary	Glossary-1
-----------------------	-------------------

Index	Index-1
--------------------	----------------

ACI Worldwide Inc.

Preface

The ***BASE24 IEP Objects Reference Manual*** describes the objects used in an Integrated Endpoint process. It provides an introduction to the Integrated Endpoint process objects and the processing they perform. It also describes the requirements for configuring these objects.

Audience

This manual is intended as an introductory guide for developers responsible for development of the Integrated Endpoint process using the integrated transaction services (ITS) architecture. It describes each of the Integrated Endpoint process objects and the processing these objects perform. The manual also serves as an informational guide for anyone who needs or desires a better understanding of Integrated Endpoint object processing. This manual only introduces the Integrated Endpoint objects. It is not intended and does not provide any instruction on object or program development.

Prerequisites

Readers of this manual should be familiar with the ***BASE24 ITS Kernel Objects Reference Manual*** and ***BASE24 IEP Configuration Manual*** before reading this manual. The ***BASE24 ITS Kernel Objects Reference Manual*** defines key concepts and terminology for the integrated transaction services architecture. The ***BASE24 IEP Configuration Manual*** defines message-based routing and its methods.

Prior knowledge of object-oriented programming methodology is not required but may also prove helpful.

Additional Documentation

The BASE24 documentation set is arranged so that each manual presents a topic or group of related topics in detail. When one manual presents a topic that has already been covered in detail in another manual, the topic is summarized and the

reader is directed to the other manual for additional information. Information has been arranged in this manner to be more efficient for readers who do not need the additional detail and at the same time provide the source for readers who require the additional information.

This manual contains references to the following BASE24 publications:

- The ***BASE24 Core Files and Tables Maintenance Manual*** contains detailed information on the VRU Configuration Data (VCD) screen and the ITS Interface Configuration File (IFCF).
- The ***BASE24 Files and Record Structures Reference Manual*** provides instructions on how to use the DDLDOC program.
- The ***BASE24 Logical Network Configuration File Manual*** contains detailed descriptions of Logical Network Configuration File (LCONF) assigns and params used by integrated endpoint objects.
- The ***BASE24 ITS Kernel Objects Reference Manual*** describes the ITS kernel objects. These objects are used when building any process in the integrated transaction services environment.
- The ***BASE24 ITS Kernel Configuration Manual*** contains detailed instructions on configuring the kernel environment.
- The ***BASE24 ITS Kernel File Maintenance Manual*** describes the ITS configuration and control screens. These screens are used when configuring or controlling any ITS process, such as the Integrated Endpoint process.
- The ***BASE24 IEP Configuration Manual*** describes the steps used to configure the Integrated Endpoint process.

In addition to the BASE24 publications, this manual contains references to the following International Organization for Standardization (ISO) publication. The 1993 ISO 8583 standard, ***Financial transaction card originated messages—Interchange message specifications***, describes the ISO messages processed by the Message ISO object.

Software

This manual documents standard processing for Integrated Endpoint Process release 1.3.

Customer-specific code modifications (that is, CSMs) may result in processing that differs from the material presented in this manual. The customer is responsible for identifying and noting these changes.

Manual Summary

The following is a summary of the contents of this manual.

“Conventions Used in This Manual” follows this preface and describes notation and documentation conventions necessary to understand the information in the manual.

Section 1, “Introduction,” provides an introduction to the Integrated Endpoint process and the objects that comprise the process.

Section 2, “Context Manager Object,” describes the Context Manager object and its role in Integrated Endpoint processing.

Section 3, “Destination Object,” describes the Destination object and its role in Integrated Endpoint processing.

Section 4, “Endpoint Class Handler Object,” describes the Endpoint Class Handler object and its role in Integrated Endpoint processing. A discussion of the generic Endpoint Class Handler object and its role in object development is also provided.

Section 5, “Endpoint Class Handler Services Object,” describes the Endpoint Class Handler Services object and its role in Integrated Endpoint processing.

Section 6, “Message ISO Object,” describes the Message ISO object and its role in Integrated Endpoint processing.

Section 7, “Integrated Endpoint Process Transaction Processing,” provides detailed step-by-step transaction processing flows for various processing situations.

A glossary that contains definitions of terms used in this manual follows the last section.

Publication Identification

Two entries appearing at the bottom of each page uniquely identify this BASE24 publication. The publication number (for example, OE-AE000-03 for the ***BASE24 IEP Objects Reference Manual***) appears on every page to assist readers in applying updates to the appropriate manual. The publication date (for example, 07/2000 for July, 2000) identifies the last time material on a particular page has

been revised. This date can be used to verify that the reader has the most current information and is particularly helpful when dealing with HELP24 to answer operating questions.

The print record, located on the page immediately following the title page, is also helpful when verifying that a manual contains the most current information. An entry is added to the print record each time a BASE24 publication is modified in any way. Each print record entry includes the date of the change, a unique version number, and the type of change (new, update, reissue, or preliminary). Version 1 is assigned to a *new* manual. Later versions can be updates or reissues, depending on the changes being made. An *update* typically involves changes on a small number of pages throughout the manual. A *reissue* occurs when all pages of the manual are replaced because changes affect a large number of pages. A manual is identified as *preliminary* when it is released to customers on a limited basis, but it has not yet reached final form.

Conventions Used in This Manual

This section explains how syntax descriptions and graphic symbols are documented in this manual. It also explains a change in Nucleus process terminology.

Syntax Notation

Syntax descriptions in this manual use several symbols and conventions to denote how commands must be entered. These conventions are described in the table below.

Notation	Meaning
UPPERCASE LETTERS	Indicate key words and literals that you must enter exactly as shown.
<i>lowercase italics</i>	Identify variables that you must provide.
Brackets []	Enclose optional syntax items that you can enter in the command.
Braces { }	Enclose a group of items from which you are required to choose one item. The items are separated within the braces by a vertical bar ().
Vertical bar	Separates alternatives in a list of items.
Ellipsis ...	Indicates the immediately preceding syntactical item can occur one or more times.
Punctuation	Must be entered as shown. This includes parentheses, commas, semicolons, and other symbols not previously described.

Notation	Meaning
Spaces	Are required as delimiters wherever it would be otherwise impossible to discriminate between adjacent words or symbols. You can insert additional spaces between any adjacent words or symbols, but not within a word or multiple character symbol.
Indented lines	Indicates a continuation of the preceding line of syntax. If the syntax of a command is too long to fit on a single line, each continuation line is indented.

Graphic Symbols

In all graphics used in this manual, processes are always depicted as rectangles or squares while objects are always depicted as circles.

Nucleus Process Terminology Change

There is one process that is the hub of an XPNET node. It manages message queues; controls all lines, stations, and processes in the system; and serves as an interprocess interface. Throughout this manual, this process is referred to as the XPNET process.

In other manuals, you may see references to the Nucleus process as performing these same functions. These two terms (Nucleus process and XPNET process) should be viewed as synonymous. The Nucleus process and the XPNET process perform the same functions and processing. As manuals are reissued, references to the Nucleus process will be replaced with references to the XPNET process.

Section 1

Introduction

The Integrated Endpoint process allows you to perform message-based and symbolic destination routing for transactions entering your system from any endpoint. Built using the integrated transaction services (ITS) architecture, the Integrated Endpoint process is composed of a number of objects. This section provides an introduction to the Integrated Endpoint process, message-based routing, symbolic destination routing, and to the objects that are part of an Integrated Endpoint process.

Integrated Endpoint Process

The Integrated Endpoint process is a satellite process that functionally, or logically, resides between the XPNET process and an application. The Integrated Endpoint process is an object-oriented process developed under the integrated transaction services (ITS) architecture. The Integrated Endpoint process maintains transaction context and the timer associated with the transaction context for certain transaction phases. The Integrated Endpoint process also determines the ultimate destinations of the transaction, optionally updates Measure counters, and manages network overload. Message destinations can be determined from the source class and message class associated with the message (message-based routing) or from the message contents and the logic built directly into the Endpoint Class Handler object (symbolic destination routing). Message-based routing is controlled by the configuration of the Destination File (DEST). Symbolic destination routing is completely controlled by the logic built into the Endpoint Class Handler object.

The Integrated Endpoint process can be used to interface with existing procedural applications as well as with ITS applications. It is not, however, required to run BASE24-atm, BASE24-pos, or BASE24-teller under the XPNET process.

Note: Code changes to existing procedural BASE24 products may be required to successfully incorporate the Integrated Endpoint process.

The Integrated Endpoint process performs the following functions. Each of these functions is described following a discussion of assumptions made regarding the Integrated Endpoint process.

- Message-based routing
- Symbolic destination routing
- Context management
- Measure counter manipulation
- Network overload management (NOM)

Assumptions

Several assumptions have been made regarding the Integrated Endpoint process. These assumptions define the capabilities for the Integrated Endpoint process and the interface requirements for existing components.

Supported Message Formats

An endpoint is any process or station from which transactions are acquired. Each endpoint has a source class and one or more message formats associated with it. Endpoint Class Handler objects can be developed for the following formats of messages, with each message format handled by a different Endpoint Class Handler object. Separate Endpoint Class Handler objects are required for each endpoint type, even if different endpoints utilize the same message format.

- Fixed structure messages with known offsets and field lengths
- Variable-length messages that use field separator (FS) characters to delimit fields
- International Organization for Standardization (ISO) 8583 bit-mapped messages

Transport Layers

All ITS processes, including the Integrated Endpoint process, run under control of the XPNET process.

Source Class

The Integrated Endpoint process uses the SOURCECLASS attribute defined in the Network Environment File (NEF) for processes and stations as a means of routing messages (message-based routing). The value from this attribute is referred to as the source class.

A process or station can have only one source class, however, any number of source classes can be supported by a single Endpoint Class Handler object.

Message-Based Routing

By examining the source and contents of a message, the Integrated Endpoint process determines the destinations for the message. The process then routes the message to each of these destinations. This is called message-based routing. The Integrated Endpoint process can route messages to any process or station in the network, including both ITS and procedural applications.

Routing Flexibility

The XPNET process currently allows for destination string routing which is similar to message-based routing, but is not as flexible in its routing logic. Destination string routing is based upon a string of data provided in the message, while message-based routing is based upon several parameters provided in the message rather than a particular string of data.

The XPNET process can also route messages to the provider of a service rather than to a specific symbolic destination. Multiple services can be configured for processes and stations running under the XPNET process. Thus, the destination determined through message-based routing can be a service of identical processes or stations.

Usage

Message-based routing is typically only needed for the request portion of a transaction. Transaction responses are automatically routed back to the originating station or process if no message-based routing is configured for the response and the application that created the response communicates with lexicons. If the application does not communicate with lexicons, the response generated by the application must be routed, just like a request, through an Integrated Endpoint process.

Message-based routing of responses is only necessary if the response is routed to a destination other than the transaction request originating station or process or if the application does not communicate with lexicons. Therefore, message-based routing for responses is typically only required when chaining messages from one ITS application to another in series, when broadcasting messages to multiple destinations in parallel, or when routing responses generated by procedural processes.

Source Class

The source of a message is represented by the source class. As described earlier, the NEF SOURCECLASS attribute configured for a process or station defines the source class for that process or station. For example, if a message is received from a host station and the SOURCECLASS attribute for the station is set to HOST, the source class associated with the message would also be HOST.

Message Class

The contents of a message are represented by a message class. The message class is typically based on the product ID, message type, and message variant associated with a message, although any combination of fields from the message can be used to derive the message class. If the source class and message class fields are not explicitly provided in the message, the Integrated Endpoint process can be constructed to determine the source or message class indirectly from the message contents. The message class consists of four subfields, with each subfield being two bytes in length.

Destination File (DEST)

The source class and message class for a message are associated with a physical destination using a routing table built in memory from the Destination File (DEST)—the Destination File extended memory table. This routing table data identifies the type of destination and how messages should be transmitted to the destination. The key to the routing table is composed from the source class, message class, and destination. If only the source class is considered for routing, all messages from a given source are routed to the same destinations. If both the source class and message class are used, different messages from a given source can be routed to different destinations. This routing mechanism is not limited to examining a single field to determine the message class for routing. Up to four subfields can be examined to determine the destinations associated with the message class.

Message-based routing involves determining the source class and message class associated with a message, comparing these values to the Destination File extended memory table, and routing the message to each of the indicated destinations. A destination can be a station, process, or a service of stations or processes.

Routing Methods

Message-based routing supports each of the following routing methods:

- Broadcast routing
- Chained routing
- Combined chained and broadcast routing
- Pass-through routing
- Simple routing

Broadcast routing allows you to route a message to multiple destinations simultaneously. Chained routing allows you to route to multiple destinations sequentially. Pass-through routing allows you to route from one station to another without accessing an application. Simple routing allows you to route a message to a single online destination. For detailed information on message-based routing methods, refer to the ***BASE24 IEP Configuration Manual***.

Symbolic Destination Routing

Symbolic destination routing allows the Integrated Endpoint process to send a message to a single symbolic destination based on the message contents and the logic built directly into the Endpoint Class Handler object for the process. The DEST is not used to determine this destination. The destination is determined solely from the message contents by the process itself. The Endpoint Class Handler object can also determine the message class for the message so the message can also be routed to additional destinations determined through message-based routing.

The Endpoint Class Handler object sends the Message Route Symbolic member function to the Endpoint Class Handler Services object to route the message to the symbolic destination. This function can also include the source class and message class associated with the message so that additional destinations can also be determined using message-based routing. The Endpoint Class Handler Services object creates an external message that is sent to the XPNET process for delivery to the specified symbolic destination. If the source class and message class are included in the function, the object also calls the Destination object to retrieve any additional destinations. The object also creates external messages and routes them for each additional destination found. Context is never associated with messages routed using the Message Route Symbolic member function.

Any desired logic can be used in the Endpoint Class Handler object to determine the symbolic destination for a message. For example, the Endpoint Class Handler object could be constructed to take advantage of destination string routing currently used with many procedural BASE24 products.

Symbolic destination routing can be useful when incorporating the Integrated Endpoint process into an existing procedural application environment where station-to-station routing (pass-through routing) is required. After an incoming message from a station has been processed by the appropriate procedural BASE24 Device Handler, Host Interface, or Interchange Interface process, it can be sent to the Integrated Endpoint process for both symbolic destination and message-based routing. Symbolic destination routing can be used to send the message to a single symbolic station destination, while message-based routing can be used to send the message to additional procedural or ITS destinations.

Context Management

Context management refers to the temporary storage of a copy of a transaction message for later use, as well as maintaining any timers associated with the message. The transaction package and its associated timers are called context. Due to the tight encapsulation of data in the ITS architecture and the context-free philosophy behind ITS server processes, context management is required outside of an ITS process. Therefore, all of the context is stored in the Integrated Endpoint process, allowing ITS server processes to run context-free. Context-free means that the server processes do not retain any information from one transaction to the next.

Thus, for all outbound transaction requests and for certain transaction responses, Integrated Authorization Server processes forward the request or response message to an Integrated Endpoint process for storage during the duration of the transaction. The context-free nature of Integrated Authorization Server processes also allows systems to be defined based on services, rather than being defined based on individual processes. Any process within the service can process any message sent to the service. The process that originally processed a transaction does not necessarily have to be the same process that completes the transaction. Any available process within the service can complete the transaction as long as the necessary context is provided by the Integrated Endpoint process.

Measure Counter Manipulation

The Integrated Endpoint process provides an interface for applications into the NET24 Performance Monitoring subsystem and performance monitoring using Measure user counters. A separate Measure user counter can be defined for each endpoint entity processed by the Integrated Endpoint process. For example, a user counter could be defined for each instance of an interface or one counter can be defined for all instances of an interface. Each Measure user counter contains an array of 15 separate counts for that endpoint entity. The following counts are defined in the Endpoint Class Handler and Endpoint Class Handler Services objects and are available for any application that requests them.

- Failed message count
- Late response count
- External authorization count and time
- Internal authorization count and time
- Authorization count and time
- Timeout count

For a detailed description of these counts, see section 4. Refer to the ***BASE24 IEP Configuration Manual*** for a discussion of Measure user counter configuration requirements and procedures.

All Measure user counters are named and configured to the Tandem Measure subsystem. Applications that need to use the counters need only provide the name of a configured Measure user counter to the Integrated Endpoint process. The counts are updated for the following messages:

- Financial transaction requests and responses
- Financial advice requests and responses
- Reversal advice requests and responses

Counts are also updated for timed out transactions and failed messages.

Network Overload Management

The network overload management capabilities of the Integrated Endpoint process provide a centralized point of control for network overload management processing on behalf of applications. Because the Integrated Endpoint process functions as the gateway into ITS-based applications, it is a natural place to provide network overload management functionality. The Integrated Endpoint process can make intelligent choices as to which messages should be discarded based on the current level of network overload and the type of message. Network overload management support in the Integrated Endpoint process prevents most ITS applications from being overloaded by new transaction requests.

The XPNET process provides the current percentage of network overload in the network header of each message delivered to the Integrated Endpoint process. The XPNET Interface object compares this percentage to the value of a Logical Network Configuration File (LCONF) param to determine whether network overload is detected. If so, The Endpoint Class Handler object can check the type of message and take appropriate action, such as discarding the message.

Messages discarded by the Integrated Endpoint process due to network overload are lost forever. It is the responsibility of the message source to resubmit a transaction request when no response is received.

Integrated Endpoint Process Objects

The Integrated Endpoint process is composed from several types of objects. Each Integrated Endpoint process contains kernel, integrated endpoint, and possibly core objects. Kernel objects provide functionality fundamental to all ITS processes. Integrated endpoint objects perform functions specific to an endpoint class. Core objects perform application-specific functions such as terminal- or interface-specific functions. Integrated endpoint objects are described in sections 2 through 6 of this manual. Kernel objects are described in the ***BASE24 ITS Kernel Objects Reference Manual***. Core objects are described in the application-specific transaction processing manual.

Each of the objects that comprise the Integrated Endpoint process are identified below.

Kernel Objects

Every Integrated Endpoint process is comprised of the following kernel objects. Each of these objects is described in detail in the ***BASE24 ITS Kernel Objects Reference Manual***.

- Application Network Services
- Class
- Event
- Process Control
- Trace
- XPNET Interface

Integrated Endpoint Objects

Every Integrated Endpoint process is comprised of the following Integrated Endpoint objects. These objects are described in sections 2 through 6.

- Context Manager object
- Destination object

- Endpoint Class Handler object
- Endpoint Class Handler Services object
- Message ISO object

Core Objects

Each Integrated Endpoint process may also contain one or more application-specific terminal or interface core objects. For more information on core objects, refer to the application-specific transaction processing manual.

Building Processes

Integrated Endpoint processes are built by binding the appropriate objects into an executable process. This is done through process MAKE files that are executed when the software is installed. Different objects are bound in for different processes, depending upon the functionality required.

Integrated Endpoint Extended Memory Tables

Some of the objects residing in the Integrated Endpoint process use extended memory tables. Each of the extended memory tables used by Integrated Endpoint process objects are described below. Detailed descriptions of how these extended memory tables are used by Integrated Endpoint process objects are provided in the individual object descriptions in sections 2 through 6 of this manual.

Depending upon the type of endpoint the Integrated Endpoint process services, the Integrated Endpoint process may also access some core extended memory tables. For more information on core extended memory tables, refer to the application-specific transaction processing manual.

Context Extended Memory Table

The Context Manager object uses the context extended memory table during transaction processing to store information about context characteristics for objects and to store and retrieve actual context information. Context extended memory table entries are built from two internal tables used by the Context Manager object—the Object Context Table and the Context Table. The context extended memory table is built during initialization of the Context Manager object.

Note: The context extended memory table is not built by the Extended Memory Table Build utility. The context extended memory table is created by the Context Manager object during initialization processing. Information is stored in and retrieved from the context extended memory table as transaction processing occurs.

Destination File Extended Memory Table

The Destination File (DEST) extended memory table is built from the Destination File (DEST) and is used for message-based routing processing. It contains exactly the same information defined in the Destination File. It is used to increase performance while the Destination object searches for message destinations based on the source class and message class of a message to be routed. The DEST extended memory table is a read-only hash table built by the Extended Memory Table Build utility.

Integrated Endpoint Processing Overview

The Integrated Endpoint process supports the processing of messages from three different sources—external from an outside entity, internal from an Integrated Server process, and internal from a procedural BASE24 process. The following paragraphs describe the processing performed by the Integrated Endpoint process for each type of message source.

External Messages

When the XPNET Interface object in the Integrated Endpoint process receives a message from the XPNET process, it calls the appropriate message handling object. The XPNET Interface object examines the source class of the message source and sends the message to the corresponding Endpoint Class Handler object for that source.

The Endpoint Class Handler object determines the message type and message class, and whether context is expected. If context is expected, the Endpoint Class Handler object builds the context key. The Endpoint Class Handler object then formats a member function and gives the message class, message source class, and, optionally, the context key to the Endpoint Class Handler Services object. If the Endpoint Class Handler object is constructed to do so, it can also determine a single symbolic destination for the message. This destination is then included in the member function sent to the Endpoint Class Handler Services object.

If context is required and it already exists for the message, the Endpoint Class Handler Services object calls the Context Manager object to retrieve context using the Context Retrieve member function. If context is required and it does not already exist for the message, the Endpoint Class Handler Services object calls the Context Manager object to store context using the Context Store member function.

If required, the Endpoint Class Handler Services object calls the Destination object to determine all message destinations associated with the source class and message class for the message. The Endpoint Class Handler Services object then formats an external message for each destination determined and gives these messages to the XPNET Interface object. The XPNET Interface object delivers each message to the XPNET process for delivery.

Internal Integrated Server Process Messages

When the XPNET Interface object in the Integrated Endpoint process receives a message from an Integrated Authorization Server process, the XPNET Interface object calls the appropriate message handler object. The XPNET Interface object determines the message destination using the source class of the message. In certain cases, such as a message sent from one Integrated Authorization Server process to another Integrated Authorization Server process, the destination of the message may be the XPNET Interface object itself. In such cases, the XPNET Interface object searches for a lexical member function in the message which is used to determine what object will handle the message.

If the message is sent to the Endpoint Class Handler Services object, the object uses additional lexicon information to route the message. The message is then routed in the same way as external messages.

Procedural BASE24 Messages

Note: The following paragraphs describe the hypothetical use of the Integrated Endpoint process with procedural BASE24 applications. This information is provided to show you how the Integrated Endpoint process could be used to route messages received from a procedural BASE24 application.

Messages received from procedural BASE24 processes would be processed by the XPNET Interface object like all other messages. The XPNET Interface object would determine the message handler the same way it does for any other message, using the source class of the message. Messages from procedural BASE24 processes would typically use an Internal Message Translate Endpoint Class Handler object to provide message classification.

The XPNET Interface object would send the message to the Internal Message Translate Endpoint Class Handler object for message classification. The Internal Message Translate Endpoint Class Handler object would then examine the standard internal message and determine the message type and message class for message-based routing. If the object is constructed to do so, it could also determine a single symbolic destination for the message.

The Internal Message Translate Endpoint Class Handler object would then call the Endpoint Class Handler Services object, passing the message class, source class, and a single symbolic destination if one is determined. The Endpoint Class Handler Services object would then perform message-based routing and route the message to all specified destinations as it would for any other Endpoint Class Handler object.

Note: Since context is already handled by the XPNET process in most procedural BASE24 applications, context typically would not be stored for messages received from a procedural BASE24 process.

Transaction Paths

Several transaction path scenarios are provided in this manual for a better understanding of the processing each object performs in the Integrated Endpoint process. Detailed step-by-step transaction path examples are provided in section 7 for the following situations:

- Transaction request authorized by an Integrated Authorization Server process
- Transaction response generated by an Integrated Authorization Server process
- Financial completion sent to an Integrated Authorization Server process
- Financial transaction authorized by a host
- Context timeout
- Network overload detected

Integrated Endpoint Process Member Functions

A member function is an action that one object can request another object to perform. Because each object is responsible for different data and for the functions related to that data, the member functions supported by each object are different.

When an object requests another object to perform a function, the object making the request is considered to be the client. The client object makes a request for services from another object. The object that performs the action is called the actor. The actor responds to the client request for service. Most objects function as either client or actor, depending on what processing is being performed.

Member functions are logically grouped into lexicons. A lexicon defines all of the member functions that a client object can request of an actor object. The lexicon name identifies the client and actor objects. For example, the Endpoint Class Handler to Endpoint Class Handler Services lexicon identifies the member functions that the Endpoint Class Handler object, as a client, can request of the Endpoint Class Handler Services object, as actor.

Some lexicons define member functions that can be sent from any object or can be received from any object. In this case, the client or actor object is generically referred to as Object in the lexicon name. For example, the Object to Destination lexicon defines the member functions that can be sent from any object to the Destination object.

The member functions that can be performed by Integrated Endpoint objects, as actors, are described in alphabetical order by lexicon on the following pages.

Note: Integrated Endpoint objects can request services from kernel objects and from application-specific (core) objects. For more information on the member functions performed by kernel objects, refer to the *BASE24 ITS Kernel Objects Reference Manual*. For more information on the member functions performed by application-specific objects, refer to the application-specific transaction processing manual.

All Lexicons

The following member function is included in all lexicons.

Version — Allows for object version synchronization between the actor and client object.

A version number is provided in the header of every member function to ensure that the request lexicon version is compatible. The minimum actor version of the client object must be equal to or less than the actor object's supported minimum version, in order for the actor object to perform the requested function. All the information required for this member function is provided in the lexicon header prefixed to all lexicon request and response messages.

Context Manager to Object Lexicon

The following member function is included in the Context Manager to Object lexicon. This function is sent from the Context Manager object to any other object in the Integrated Endpoint process.

Context Timeout — Indicates that the timer associated with message context has expired before the associated message acknowledgment is received.

The Context Timeout member function is sent from the Context Manager object to any object (typically, the Endpoint Class Handler Services object) when the timer associated with the context for a message response expires before the associated acknowledgment is received. The Context Manager object is informed of the timeout when it receives a System Time Signal member function from the XPNET Interface object before context is retrieved.

Endpoint Class Handler to Endpoint Class Handler Services Lexicon

The following member functions are included in the Endpoint Class Handler to Endpoint Class Handler Services lexicon. These functions can be sent from the Endpoint Class Handler object to the Endpoint Class Handler Services object.

Message Route — Routes a message that does not have any context associated with it using message-based routing.

The Message Route member function is sent from the Endpoint Class Handler object to the Endpoint Class Handler Services object to route a message that does not have any context associated with it using message-based routing. The Endpoint Class Handler Services object calls the Destination object to retrieve all of the destinations for the source class and message class associated with the message.

Message Route Context Retrieve — Routes messages with previously stored context using message-based routing.

The Message Route Context Retrieve member function is sent from the Endpoint Class Handler object to the Endpoint Class Handler Services object to route a message with stored context using message-based routing. The Endpoint Class Handler Services object retrieves the context by calling the Context Manager object and routes the message to the appropriate destinations. The Endpoint Class Handler Services object calls the Destination object to retrieve all of the destinations for the message.

Message Route Symbolic — Routes a message to a single symbolic destination specified in the request as well as to additional destinations determined through message-based routing. No context is associated with the message.

The Message Route Symbolic member function is sent from the Endpoint Class Handler object to the Endpoint Class Handler Services object to route a message without any context to a single symbolic destination as well as to additional destinations determined through message-based routing. The Endpoint Class Handler Services object calls the Destination object to retrieve all of the additional destinations for the source class and message class associated with the message. If the source class and message class are not provided in the request, message-based routing is not performed.

Register — Registers the Endpoint Class Handler object message context requirements with the Endpoint Class Handler Services object.

The Register member function is sent from an Endpoint Class Handler object to the Endpoint Class Handler Services object as an interim step in registering the Endpoint Class Handler object context requirements with the Context Manager object. This function specifies the following:

- The maximum number of bytes needed to store a block of context
- The maximum number of bytes needed to store a context key
- The number of blocks to initially allocate on behalf of the object
- The number of blocks to be allocated for additional storage if the primary allocation is exceeded

When this request is received, the Endpoint Class Handler Services object formats another Register member function and sends it to the Context Manager object. This request contains all the information supplied in the Register member function from the Endpoint Class Handler object along with the additional data space required by the Endpoint Class Handler Services object.

Endpoint Class Handler Services to Object Lexicon

The following member functions are included in the Endpoint Class Handler Services to Object lexicon. These functions can be sent from the Endpoint Class Handler Services object to any object.

External Message — Contains an external message received for which there is no previously stored context.

The External Message member function is sent from the Endpoint Class Handler Services object to an object in an Integrated Server process when a message is received from an external entity and there is no context associated with the message. The request indicates the symbolic name of the station or process from which the message originated and contains a flag indicating whether the request was enveloped within a Transport Layer Object to Transport Layer Object lexicon, Message to Object request. This request also indicates the time the message was acquired by the Integrated Endpoint process, the offset from the beginning of the lexicon request to the external message, and the length of the external message.

External Message with Context — Contains an external message received for which there is previously stored context.

The External Message with Context member function is sent from the Endpoint Class Handler Services object to another object in an Integrated Server process when a message is received from an external entity and context has been previously stored. The request indicates the symbolic name of the station or process from which the message originated and contains a flag indicating whether the request was enveloped within a Transport Layer Object to Transport Layer Object lexicon, Message to Object request. This request also indicates the time the message was acquired by the Integrated Endpoint process, the offset from the beginning of the lexicon request to the external message, the length of the external message, the offset from the beginning of the lexicon request to the context, and the length of the context.

External Message Failed — Contains an external message, with no previously stored context, that failed to be delivered to an external destination.

The External Message Failed member function is sent from the Endpoint Class Handler Services object to another object in an Integrated Server process when a message that was sent to an external entity from the Integrated Endpoint process could not be delivered by the XPNET process. No previously stored context exists for the message. The request indicates the symbolic name of the station or process

from which the message originated and contains a flag indicating whether the request was enveloped within a Transport Layer Object to Transport Layer Object lexicon, Message to Object request. This request also contains a code indicating the reason the message could not be delivered to its intended destination, the offset from the beginning of the lexicon request to the external message, and the length of the external message.

External Message Failed with Context — Contains an external message, with previously stored context, that failed to be delivered to an external destination.

The External Message with Context member function is sent from the Endpoint Class Handler Services object to another object in an Integrated Server process when a message that was sent to an external entity could not be delivered by the XPNET process and context had been previously stored for the message. The request indicates the symbolic name of the station or process from which the message originated and contains a flag indicating whether the request was enveloped within a Transport Layer Object to Transport Layer Object lexicon, Message to Object request. This request also contains a code indicating the reason the message could not be delivered to its intended destination, the offset from the beginning of the lexicon request to the external message, the length of the external message, the offset from the beginning of the lexicon request to the context, and the length of the context.

External Message Timeout — Contains the previously stored context that timed out for an external message before the associated acknowledgment or response message was received.

The External Message Timeout member function is sent from the Endpoint Class Handler Services object to another object in an Integrated Server process when the context timer for an external message expires. This request indicates the offset from the beginning of the lexicon to the context, and the length of the context.

External Message Late — Contains an external message that was received, but no context could be retrieved because the original response timed out.

The External Message Late member function is sent from the Endpoint Class Handler Services object to another object in an Integrated Server process when a response is received from an external entity, but no context can be found because the original response timed out. The request indicates the symbolic name of the station or process from which the message originated and contains a flag indicating whether the request was enveloped within a Transport Layer Object to Transport Layer Object lexicon, Message to Object request. This request also

contains the time the message entered the application, the offset from the beginning of the lexicon request to the external message, and the length of the external message.

Message ISO to Object Lexicon

The following member functions are included in the Message ISO to Object lexicon. These functions can be sent from the Message ISO object to any object.

MAC Generate/PIN Translate — Generates a message authentication code (MAC) or translates a PIN for an external message received.

The MAC Generate/PIN Translate member function is sent from the Message ISO object in the Integrated Endpoint process to an ISO endpoint object in an Integrated Authorization Server process to generate a message authentication code (MAC) or to translate a PIN for the external message.

MAC Verify — Verifies a message authentication code (MAC) provided in an external message received.

The MAC Verify member function is sent from the Message ISO object in the Integrated Endpoint process to an ISO endpoint object in an Integrated Authorization Server process to verify a message authentication code (MAC) provided in the external message received.

Message Compress Unique — Compresses any unique fields in an external message to be sent. A unique field is one that is specific to an application rather than to the ISO standard.

The Message Compress Unique member function is sent from the Message ISO object to an ISO endpoint object to compress any unique fields in the external message that are application-specific. These fields are compressed from a fixed Internal Transaction Data (ITD) format back into an external bit-mapped ISO format.

Message Expand Unique — Expands any unique fields in an external message received. A unique field is one that is specific to an application rather than to the ISO standard.

The Message Expand Unique member function is sent from the Message ISO object to an ISO endpoint object to expand any unique fields in the Internal Transaction Data (ITD) message that are application-specific. These fields are expanded from a bit-mapped ISO format into a fixed Internal Transaction Data (ITD) format.

Message Trace Unique — Traces any unique fields in an external message received. A unique field is one that is specific to an application rather than to the ISO standard.

The Message Trace Unique member function is sent from the Message ISO object to an ISO endpoint object to trace any unique fields in the external message that are application-specific.

Object to Context Manager Lexicon

The following member functions are included in the Object to Context Manager lexicon. These functions can be sent from any object to the Context Manager object.

Context Retrieve — Retrieves previously stored context and deletes the timer associated with the context.

The Context Retrieve member function is sent from any object (typically, the Endpoint Class Handler Services object) to the Context Manager object to retrieve previously stored context and delete the timer associated with the context. The Context Manager object typically stores context and sets a timer when a new response is received from an Integrated Authorization Server process. Context retrieval typically occurs when the associated acknowledgment is returned to the Integrated Endpoint process from the originating endpoint.

Context Store — Stores context and sets a timer associated with the context.

The Context Store member function is sent from any object (typically, the Endpoint Class Handler Services object) to the Context Manager object to store context associated with a message (for example, a transaction response) and to start a timer associated with the message. The Context Manager object retrieves context and deletes the timer when the associated message (for example, an acknowledgment to the response) is received.

Register — Registers an object with the Context Manager object. An object must register with the Context Manager object before it can request to store and retrieve message context.

The Register member function is sent from another object to the Context Manager object to register the calling object with the Context Manager object.

Object to Destination Lexicon

The following member function is included in the Object to Destination lexicon. This function is sent from any object to the Destination object.

Object ID Retrieve (Destination Retrieve) — Retrieves the symbolic process name and object ID destinations to which a message received from an endpoint will be sent from the Destination File extended memory table.

The Object ID Retrieve member function is sent from any object (typically the Endpoint Class Handler Services object) to the Destination object to retrieve all the destinations from the Destination File extended memory table to which a given message received from an endpoint should be sent. This request contains both the source class and message class associated with the message, the key value of the last destination returned by the Destination object, the address of the last destination returned by the Destination object, and a series of flags indicating which message class subfields can use wildcard characters, if any, for the search of the Destination File extended memory table. Parts of the message class can use wildcard characters when searching the table and more than one destination can be retrieved if broadcast routing is being used.

Object to Endpoint Class Handler Services Lexicon

The following member functions are included in the Object to Endpoint Class Handler Services lexicon. These functions can be sent from any object to the Endpoint Class Handler Services object.

Message — Contains a message to be routed to an external entity for which no context needs to be stored.

The Message member function is sent from any object (typically, an object in an Integrated Authorization Server process) to the Endpoint Class Handler Services object to route a message to an external entity.

Message Context Store — Contains a message to be routed to an external entity for which context needs to be stored.

The Message Context Store member function is sent from any object (typically, an object in an Integrated Authorization Server process) to the Endpoint Class Handler Services object when a message needs to be routed to an external entity and context needs to be stored.

Object to Message ISO Lexicon

The following member functions are included in the Object to Message ISO lexicon. These functions can be sent from any object to the Message ISO object.

Message Compress — Compresses a fixed Internal Transaction Data (ITD) formatted message back into an external bit-mapped ISO formatted message.

The Message Compress member function is sent from any object to the Message ISO object to compress an Internal Transaction Data (ITD) message back into an ISO external message. Message compression only occurs with Message ISO objects residing in Integrated Authorization Server processes when a message response is about to be sent back to its originating endpoint. Message compression is the reverse of message expansion, which occurs when a message needs to be converted from a bit-mapped ISO format into a fixed ITD format. Message expansion is performed by the Message ISO object residing in an Integrated Endpoint process.

Message Expand — Expands a bit-mapped ISO formatted message into a fixed Internal Transaction Data (ITD) formatted message.

The Message Expand member function is sent from any object to the Message ISO object to expand an ISO external message into Internal Transaction Data (ITD) format. Message expansion only occurs with Message ISO objects residing in Integrated Endpoint processes when an external request message is received from an ISO endpoint. Message expansion is the reverse of message compression, which occurs when a message needs to be converted from the ITD format back into external ISO format.

Destination File Documentation

The record structure for the Destination File (DEST) is documented online using comments in the Data Definition Language (DDL) source schema files that define the structure. The DEST record structure DDLs are located on the OE13DICT subvolume.

To improve the readability and usability of the online information, ACI provides a utility program called DDLDOC. The DDLDOC utility retrieves information from the compiled data dictionary and formats the information as the user requests. The information provided by the DDLDOC utility includes the position of the field in the file, the field name, and the data type of the field. Field names can be shown in COBOL or TAL format. Detailed field descriptions, when requested by the user, are printed below the field name and data type information.

The name of the DDL record structure for the Destination File is DEST.

The OE13DICT subvolume also contains a record structure for the lexicon message header (LEX-HDR).

For more information on using the DDLDOC utility, see the ***BASE24 Files and Record Structures Reference Manual***.

ACI Worldwide Inc.

Section 2

Context Manager Object

The Context Manager object provides all context management functions required for the Integrated Endpoint process. This includes the storage and retrieval of message context and the management of timers associated with the stored context. This section describes the Context Manager object and includes the following topics:

- An overview of how the Context Manager object fits into message processing
- A discussion of the normal processing performed by the Context Manager object
- A discussion of the configuration requirements for the Context Manager object
- A list of the member functions supported by the Context Manager object

Overview

In the Integrated Endpoint process, the Context Manager object maintains the transaction context for a transaction. Context is considered to be a copy of a message and a timer associated with the message.

Storage of message context in the Integrated Endpoint process allows Integrated Authorization Server processes to run context-free and allows for services of Integrated Authorization Server processes to be defined rather than having independent Integrated Authorization Server processes. For more information on the context-free nature of Integrated Authorization Server processes, refer to section 1.

The storage and retrieval of message context is determined by the Endpoint Class Handler object for messages received from an object or by the Endpoint Class Handler Services object for messages received from an Integrated Authorization Server process.

Context Storage

Context needs to be stored for any message sent from an Integrated Authorization Server process for which an associated message is expected immediately in return. This includes transaction requests that are sent to an external issuer and transaction responses that are sent to an external acquirer. Context is not stored for response messages sent to an issuer, since the original transaction context is the responsibility of the issuer, not the Integrated Authorization Server process that authorized the transaction. Context is also not stored for requests received from an external acquirer for the same reason. The Integrated Authorization Server process always indicates whether context needs to be stored for every message it sends to the Integrated Endpoint process.

Note: Context is also not stored for requests sent to procedural issuer host or interchange interface processes or for responses sent to procedural acquirer host or interchange interface processes. Storing the context in these situations would duplicate the context management functions already performed by the procedural host or interchange interface process.

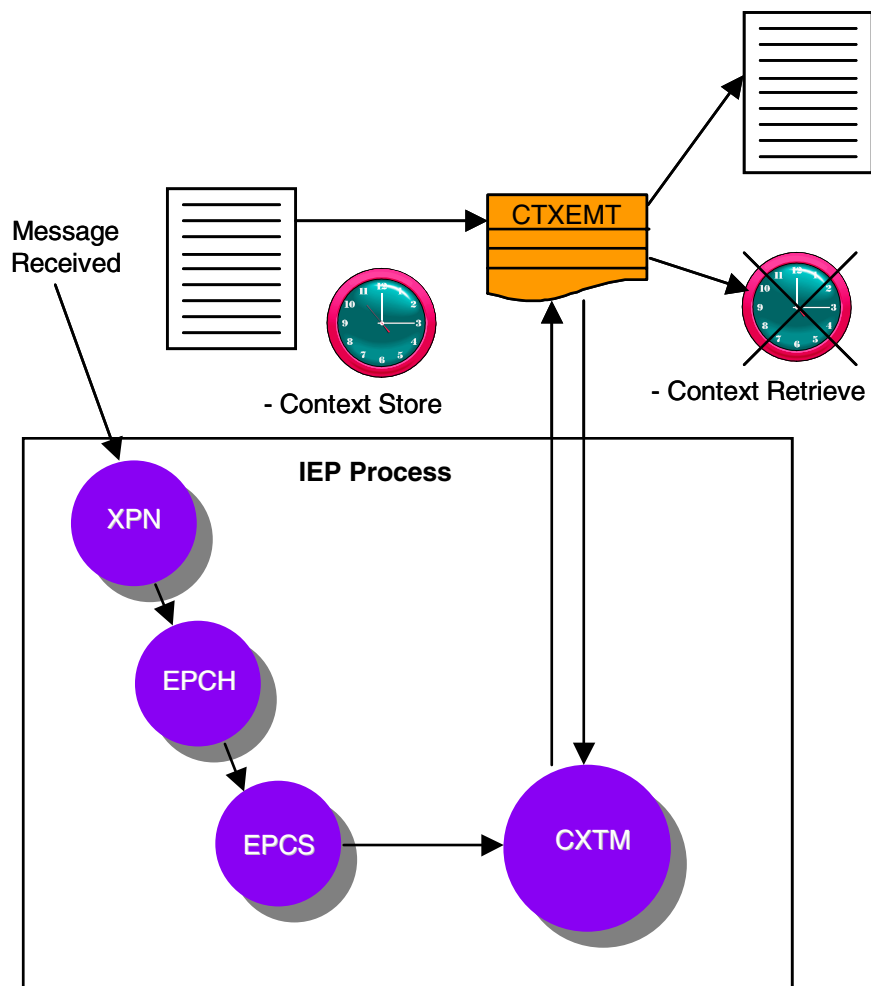
Context Retrieval

Message context allows the transaction acquirer to reconstruct a transaction message quickly without having to gather all the information that was used to build the message originally. Message context is retrieved both when the associated message is received on time and when the context times out before the associated message is received. When the associated message is received on time, context is retrieved to complete and log the transaction. When the context times out before the associated message is received, the context can be retrieved to reconstruct the original request and send it to an alternate authorizer. If a late response is received after the context times out, the late response is logged as an exception.

Context Processing

For messages that require context, the object stores a copy of the message and sets a timer associated with the message. Transaction context is then retrieved when the associated message is received or the context times out, whichever occurs first. Both the copy of the message and the timer associated with the message are considered to be context. The context is maintained until the expected associated message is received. At this time the context of the message is retrieved and the timer associated with the message is deleted. If a context timer expires before the associated message is received, the context is immediately retrieved and returned to the message originator.

The role of the Context Manager object in Integrated Endpoint process transaction processing is depicted in the illustration on the following page. A key to the acronyms used in the illustration is provided immediately following the illustration.



Key

- CTXM = Context Manager object
- CTXEMT = Context extended memory table
- EPCH = Endpoint Class Handler object
- EPCS = Endpoint Class Handler Services object
- IEP = Integrated Endpoint
- XPN = XPNET Interface object

Normal Processing

This subsection describes the normal processing performed by the Context Manager object.

Context Extended Memory Table

The Context Manager object uses the context extended memory table during transaction processing to store information about context characteristics for objects as well as to store and retrieve the actual context information. The context extended memory table is allocated as an exclusive read and write extended memory segment during object initialization. The contents of the context extended memory table segment are continually modified as transaction processing occurs. Context extended memory table entries are from two tables—the Object Context Table and the Context Table described below.

The Object Context Table is a linked list that specifies the context storage requirements needed on behalf of an object. Entries in this table are added when objects register with the Context Manager object using the Register member function.

The Context Table consists of segmented entries that contain the actual message context and the timer associated with the context. Each segment is controlled by an entry in the Object Context table. The Context Manager object adds a segment to this table when it receives the Context Store member function. The object removes a segment from this table when it receives the Context Retrieve member function or when a timeout occurs (in which case, the object receives the System Time Signal member function from the XPNET Interface object).

Context Storage and Retrieval

Any time an object (typically, the Endpoint Class Handler object in the Integrated Endpoint process or a core object in an Integrated Authorization Server process) needs to store or retrieve context, it calls the Context Manager object with a Register member function. Upon receipt of the Register member function, the Context Manager object adds the object to the context extended memory table and calculates the amount of space needed to store the context in the table. The information needed to determine the space requirements is supplied in the CTX-BLK-SIZE, CTX-KEY-SIZE, ADNL-DATA-BLK-SIZE, CTX-BLK-CNT fields of the Register member function. The context extended memory table registers an object prior to storing or retrieving context on behalf of the object. Each entry in

the table contains information about the object's context requirements. The context extended memory table contains the actual context being stored along with time and response information.

The Context Manager object uses two Logical Network Configuration File (LCONF) params for context storage. The CTXM-SWAPVOL assign identifies the Tandem volume on which the object creates a swap file for data held in memory. The swap file is used only when available memory space becomes unavailable. The CTXM-SWAPVOL assign defaults to the volume on which the Context Manager object executable resides. The CTXM-CONTEXT-PAGES param indicates the maximum number of pages of context that can be stored in memory at any one time. A page is equivalent to 4096 bytes. This param should be set to a value large enough to hold as many pages of context as anticipated for the Integrated Endpoint process during peak processing periods. Both the CTXM-SWAPVOL assign and the CTXM-CONTEXT-PAGES param are described in the *BASE24 Logical Network Configuration File Manual*.

Context Storage

All requests to store context contain the endpoint class handler identifier, the context itself, and the key to the context. The Context Manager object stores the transaction context in the context extended memory table, according to the key provided. The Context Manager object retrieves an entry from the free list and places it on the used list.

The Context Manager object determines whether the free list from the table is empty. If the free list is empty, the Context Manager object enlarges its extended segment. The additional space is used to store the context for the current request and then is placed on the used list.

Context Retrieval

Context can either be retrieved by context key or by context address.

The first method of context retrieval is used when the object receives the Context Retrieve member function. The request contains the endpoint class handler identifier and the key to the context. The Context Manager object retrieves the context, deletes the associated timer, and returns the context in the member function response. The recipient of the context, then uses the context to reconstruct the original message information without having to gather it a second time. If the requested context cannot be found, the Context Manager object returns an error status code.

The second method of retrieval is used when a timeout condition occurs. The Context Manager object retrieves the context by the address provided and forwards it to the Endpoint Class Handler Services object.

Timeout Processing

If a message timer expires before an associated message (for example, a completion) is received, the Context Manager object formats the Context Timeout member function and sends it to the Endpoint Class Handler Services object. The Context Manager object also forwards the message context to the Endpoint Class Handler Services object so it can be returned to the message originator. The duration of time context is stored and the object to be notified if the context times out are specified by the calling application when the context is stored. The Context Manager object is notified of a timeout when the System Time Signal member function is received from the XPNET Interface object.

STATUS Command

When a STATUS command is sent to the Context Manager object, the Context Manager object returns the status of each entry in the Object Context Table, the total number of pages currently used, and the name of the swap file currently used.

Configuration

This subsection lists the configuration requirements for the Context Manager object.

Kernel Configuration

The Context Manager object must be configured to every Integrated Endpoint process running in the ITS environment.

A record must be added to the Object File (OBJ) for the Context Manager object. Once this OBJ record has been established, records associated with the object must be added to the Process/Object Relationship File (PORF). For detailed instructions on configuring the kernel environment, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

LCONF Assigns and Params

The Context Manager object uses the CTXEMT-SWAPVOL assign and the CTXEMT-CONTEXT-PAGES param from the Logical Network Configuration File (LCONF) in its processing. For descriptions of this assign and param and for instructions on adding them to the LCONF, refer to the ***BASE24 Logical Network Configuration File Manual***.

Member Functions

This subsection documents the member functions sent and received by the Context Manager object. Integrated endpoint member functions are described in section 1. Kernel member functions are described in the *BASE24 ITS Kernel Objects Reference Manual*.

Member Functions Sent

The table below shows the member functions sent by the Context Manager object, arranged according to the lexicon of which they are a part.

Lexicon Name	Member Function
All lexicons	Version
Context Manager to Object	Context Timeout
Object to Application Network Services Object	Register ANS
Object to Space	Memory Get
Object to Trace	Add Trace

Member Functions Received

The table below shows the member functions received by the Context Manager object, arranged according to the lexicon of which they are a part.

Lexicon Name	Member Function
All lexicons	Version
Application Network Services Object to Object	Initialize
	Notify
	System Message
	Stop ANS

Lexicon Name	Member Function
Object to Context Manager	Context Retrieve
	Context Store
	Register
Process Control to Object	Status
TLO to Object	System Time Signal
Trace to Object	Start Trace
	Stop Trace

Section 3

Destination Object

The Destination object determines the destinations for messages received from an endpoint based on the source class and message class associated with a message. This is known as message-based routing. This section describes the Destination object and includes the following topics:

- An overview of how the Destination object fits into message processing by the Integrated Endpoint process
- A detailed discussion of the destination retrieval processing performed by the Destination object
- A discussion of the configuration requirements for the Destination object
- A list of the member functions supported by the Destination object

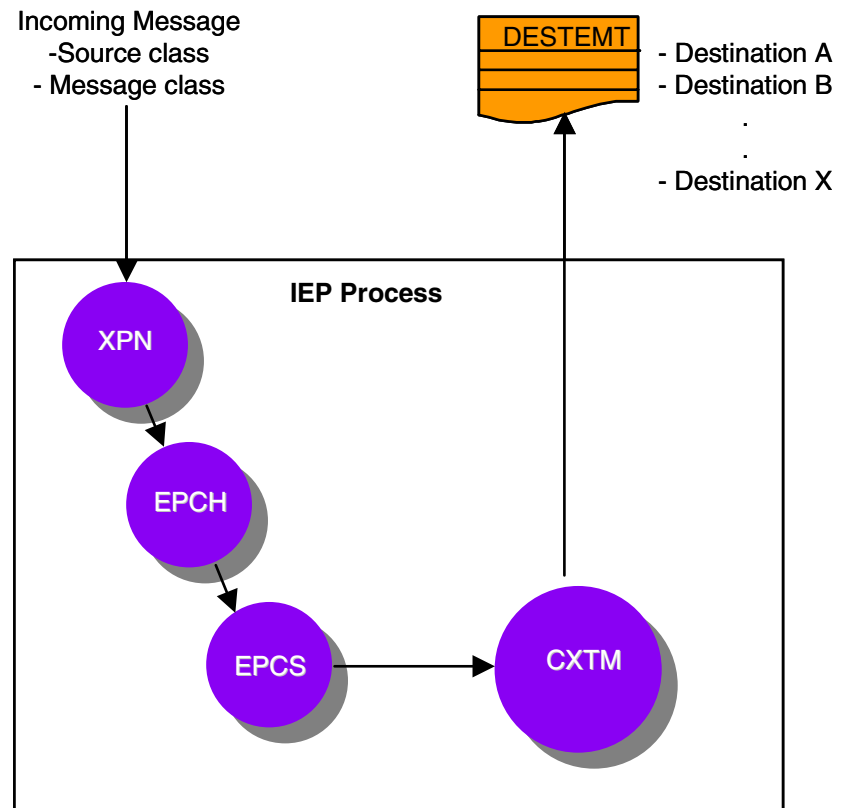
Overview

The Destination object maintains destination data for messages routed through the Integrated Endpoint process using message-based routing. The Destination object can be called by the Endpoint Class Handler Services object, on behalf of the Endpoint Class Handler object or a core object to determine the destinations for a particular message. When called, the Destination object retrieves the destinations for the source class and message class associated with the message from the Destination File extended memory table. More than one destination record can exist for a particular source class and message class combination if the message is broadcast to multiple destinations.

The Destination object opens the Destination File extended memory table in shared exclusion mode and read-only access mode. The Destination object does not modify the Destination File extended memory table in any way. It simply reads what is currently defined in the table and returns that information to the client object. The Destination object retrieves all destinations for the source class and message class of a message received from an endpoint. The message is then routed to each destination found by the Endpoint Class Handler Services object.

Destination File extended memory table entries are built from Destination File (DEST) records and are sorted by source class and then by message class. The message class can take a number of different forms, depending on how you have defined the message class for your environment. When a change is made to the Destination File, the Destination File extended memory table must be rebuilt using the Extended Memory Table Build utility. The Destination object then must reallocate the table using the EMT WARMBOOT or LCONF WARMBOOT command before the change will take effect in processing. The Destination File extended memory table is a read-only hash table built by the Extended Memory Table Build utility. For more information on the Extended Memory Table Build utility, see the *BASE24 ITS Kernel Configuration Manual*.

The functions of the Destination object are depicted in the illustration on the following page. A key to the abbreviations used in the illustration is provided immediately following the illustration.

**Key**

DEST	= Destination object
DESTEMT	= Destination File (DEST) extended memory table
EPCH	= Endpoint Class Handler object
EPCS	= Endpoint Class Handler Services object
IEP	= Integrated Endpoint
XPN	= XPNET Interface object

Normal Processing

This subsection describes the normal processing performed by the Destination object.

Initialization

During initialization, the Destination object obtains frame space and initializes its state. When an object initializes its state, this means it initializes its internal data to default values or to values obtained from Logical Network Configuration File (LCONF) assigns and params. The object also allocates the Destination File extended memory table during initialization.

Destination Retrieval

The Destination object retrieves destinations from the Destination File extended memory table when it receives the Destination Retrieve member function. The Destination object searches the Destination File extended memory table based on the source class and message class included in the request. The source class is always the first part of the key used to search the table. Because customers can derive benefit from grouping certain message classes together for routing purposes, the message class contained in the second portion of the key used to search the Destination File extended memory table can take a number of different formats. To support the functionality of grouping message classes into logical groups for the purpose of determining a message's destination, ACI has defined a hierarchy of four subfields for the MESSAGE CLASS field. This hierarchy has been defined to indicate the various ways the message class can be formatted for searching the Destination File extended memory table. This hierarchy also defines the order in which the table should be searched using the various message class values. For a detailed description of the MESSAGE CLASS field and its subfields, see the Destination File (DEST) description in the ***BASE24 IEP Configuration Manual***.

Wildcard Routing

The MESSAGE CLASS field can contain wildcard values. The following table, referred to as the wildcard combinations table, shows you how the message class subfields can be manipulated with the wildcard character for searching the Destination File extended memory table. Wildcard values are denoted by asterisks (*). Each row in the table represents a different step in search processing. The

wildcard flag values table beginning on the following page refers to the steps provided in this table for the search processing performed for each possible wildcard combination.

Step	Field-1	Field-2	Field-3	Field-4
1	Value	Value	Value	Value
2	Value	Value	Value	**
3	Value	Value	**	Value
4	Value	Value	**	**
5	Value	**	Value	Value
6	Value	**	Value	**
7	Value	**	**	Value
8	Value	**	**	**
9	**	Value	Value	Value
10	**	Value	Value	**
11	**	Value	**	Value
12	**	Value	**	**
13	**	**	Value	Value
14	**	**	Value	**
15	**	**	**	Value
16	**	**	**	**

Search Key

The Destination object uses the message class wildcard flag field along with the message class field (msg-clas) in the Destination Retrieve member function to build a base message class field value used for the search key. The base message class field contains the message class value the object uses when initially searching

the Destination File extended memory table. The object can also search for other combinations of message class subfields, depending on the wildcard options used for the base message class field.

The message class wildcard flag field is determined by the Endpoint Class Handler object from the application it supports. For example, the Voice Response Unit Endpoint Class Handler object obtains the value of this field by interface instance from the Interface extended memory table. This table is built from values entered on the VRU Configuration Data (VCD) screen and stored in the ITS Interface Configuration File (IFCF). The values of this field correspond to the 16 possible ways in which the wildcard character can be used in the base message class field for searches of the Destination File extended memory table (see the wildcard combinations table on the previous page). Setting the subfields in the base message class field to certain values dictates which steps in the wildcard combinations table the Destination object will perform. The following table, referred to as the wildcard flag values table, indicates which steps the Destination object will take for each possible value in the message class wildcard flag field of the Destination Retrieve member function. The message class wildcard flag field values shown here are the binary values stored in the IFCF. Each value indicates a different combination of wildcard values for the four message class subfields. The steps listed in this table correspond to steps in the wildcard combinations table.

For more information on the VCD and IFCF, refer to the ***BASE24 Core Files and Tables Maintenance Manual***.

Wildcard Flag Value	Processing
0	Set all subfields in the base message class field to the values in the corresponding subfields in the msg-clas field in the request. No wildcard values are used. This is equivalent to searching at step 1 only in the wildcard combinations table.
1	Set Field-4 in the base message class field to wildcard values. Set Field-1, Field-2, and Field-3 in the base message class field to the values in the corresponding subfields in the msg-clas field of the request. This is equivalent to searching at steps 2, 4, 6, 8, 10, 12, 14, and 16 in the wildcard combinations table.

Wildcard Flag Value	Processing
2	Set Field-3 in the base message class field to wildcard values. Set Field-1, Field-2, and Field-4 in the base message class field to the values in the corresponding subfields in the msg-clas field in the request. This is equivalent to searching at steps 3, 4, 7, 8, 11, 12, 15, and 16 in the wildcard combinations table.
3	Set Field-3 and Field-4 in the base message class field to wildcard values. Set Field-1 and Field-2 in the base message class field to the values in the corresponding subfields in the msg-clas field in the request. This is equivalent to searching at steps 4, 8, 12, and 16 in the wildcard combinations table.
4	Set Field-2 in the base message class field to wildcard values. Set Field-1, Field-3, and Field-4 in the base message class field to the values in the corresponding subfields in the msg-clas field in the request. This is equivalent to searching at steps 5, 6, 7, 8, 13, 14, 15, and 16 in the wildcard combinations table.
5	Set Field-2 and Field-4 in the base message class field to wildcard values. Set Field-1 and Field-3 in the base message class field to the values in the corresponding subfields in the msg-clas field in the request. This is equivalent to searching at steps 6, 8, 14, and 16 in the wildcard combinations table.
6	Set Field-2 and Field-3 in the base message class field to wildcard values. Set Field-1 and Field-4 in the base message class field to the values in the corresponding subfields in the msg-clas field of the request. This is equivalent to searching at steps 7, 8, 15, and 16 in the wildcard combinations table.

Wildcard Flag Value	Processing
7	Set Field-2, Field-3, and Field-4 in the base message class field to wildcard values. Set Field-1 in the base message class field to the value in the corresponding field in the msg-clas field of the request. This is equivalent to searching at steps 8 and 16 in the wildcard combinations table.
8	Set Field-1 in the base message class field to wildcard values. Set Field-2, Field-3, and Field-4 in the base message class field to the value in the corresponding field in the msg-clas field of the request. This is equivalent to searching at steps 9, 10, 11, 12, 13, 14, 15, and 16 in the wildcard combinations table.
9	Set Field-1 and Field-4 in the base message class field to wildcard values. Set Field-2 and Field-3 in the base message class field to the value in the corresponding field in the msg-clas field of the request. This is equivalent to searching at steps 10, 12, 14, and 16 in the wildcard combinations table.
10	Set Field-1 and Field-3 in the base message class field to wildcard values. Set Field-2 and Field-4 in the base message class field to the value in the corresponding field in the msg-clas field of the request. This is equivalent to searching at steps 11, 12, 15, and 16 in the wildcard combinations table.
11	Set Field-1, Field-3, and Field-4 in the base message class field to wildcard values. Set Field-2 in the base message class field to the value in the corresponding field in the msg-clas field of the request. This is equivalent to searching at steps 12 and 16 in the wildcard combinations table.

Wildcard Flag Value	Processing
12	Set Field-1 and Field-2 in the base message class field to wildcard values. Set Field-3 and Field-4 in the base message class field to the value in the corresponding field in the msg-clas field of the request. This is equivalent to searching at steps 13, 14, 15, and 16 in the wildcard combinations table.
13	Set Field-1, Field-2, and Field-4 in the base message class field to wildcard values. Set Field-3 in the base message class field to the value in the corresponding field in the msg-clas field of the request. This is equivalent to searching at steps 14 and 16 in the wildcard combinations table.
14	Set Field-1, Field-2, and Field-3 in the base message class field to wildcard values. Set Field-4 in the base message class field to the value in the corresponding field in the msg-clas field of the request. This is equivalent to searching at steps 15 and 16 in the wildcard combinations table.
15	Set all subfields in the base message class field to wildcard values. This is equivalent to searching at step 16 only in the wildcard combinations table.
Any other value	Raise an exception.

Search Processing

Once the base message class field value is calculated, the Destination object searches the Destination File extended memory table using this search key. If no entries are found to match this initial search key, then the key structure defined in the next possible wildcard combination is used to search the Destination File extended memory table.

When building the search key for each possible wildcard combination, only one subfield in the message class portion of the key is changed from the value in the preceding step. If the value to be placed into the one subfield being changed is

equivalent to the value that was already in the subfield, the Destination object skips to the next possible step in the wildcard combinations table. The search of the Destination File extended memory table continues until all possible combinations of wildcard values for the search key are exhausted. Multiple records can be found for any given search key if broadcast routing is used. Once all possible matching records for a search key are found, the Destination object searches through the rest of the possible search keys for the given wildcard flag value to find any other entries that match the search criteria.

With each matching record found, the Destination object returns the symbolic name of the destination, the message handler object if applicable to the destination, and other destination-specific information. This information is returned in the Destination Retrieve member function response sent back to the Endpoint Class Handler Services object.

If the number of destinations found during the search reaches the maximum number of responses that will fit in the Destination Retrieve member function response, the Destination object records where it left off in the search and sets the status in the response to indicate that more destinations may exist. When the Endpoint Class Handler Services object receives a response indicating that more destinations may exist, it formats a subsequent Destination Retrieve member function and sends it to the Destination object. The Destination object will then begin searching where it previously left off.

Configuration

This subsection lists the configuration requirements for the Destination object.

Kernel Configuration

The Destination object must be configured to every Integrated Endpoint process running in the ITS environment.

A record must be added to the Object File (OBJ) for the Destination object. Once this OBJ record has been established, records associated with the object must be added to the following files. For detailed instructions on configuring the kernel environment, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

- Process/Object Relationship File (PORF)
- EMT/Object Relationship File (EORF)

LCONF Assigns and Params

The Destination object uses the DESTEMT assign from the LCONF in its processing. For a description of this assign and for instructions on adding it to the LCONF, refer to the ***BASE24 Logical Network Configuration File Manual***.

The DESTEMT can be reallocated using the EMT WARMBOOT or LCONF WARMBOOT command.

Member Functions

This subsection lists the member functions sent and received by the Destination object. Integrated Endpoint member functions are described in section 1. Kernel member functions are described in the *BASE24 ITS Kernel Objects Reference Manual*.

Member Functions Sent

The table below shows the member functions sent by the Destination object, arranged according to the lexicon of which they are a part.

Lexicon Name	Member Function
All lexicons	Version
Object to Application Network Services Object	Register
Object to Event	Log
Object to Space	Memory Get
Object to Trace	Add Trace

Member Functions Received

The table below shows the member functions received by the Destination object, arranged according to the lexicon of which they are a part.

Lexicon Name	Member Function
All lexicons	Version
Application Network Services Object to Object	Initialize
	Stop ANS
Object to Destination	Destination Retrieve

Lexicon Name	Member Function
Process Control to Object	EMT Warmboot
	LCONF Warmboot
Trace to Object	Start Trace
	Stop Trace

ACI Worldwide Inc.

Section 4

Endpoint Class Handler Object

The Endpoint Class Handler object identifies the message class associated with a message received from a particular type of endpoint, or source class, and determines whether context for the message needs to be retrieved. The Endpoint Class Handler object calls other objects within the Integrated Endpoint process as needed to fulfill these functions. The Endpoint Class Handler object can also determine a single symbolic destination for the message if it is constructed to do so. The message to be processed is always either a normal message that originated from the endpoint, a failed message that originated from the endpoint, or a station link up or down message received from the XPNET process.

One Endpoint Class Handler object is required for each type of endpoint class serviced by the Integrated Endpoint process. Each object handles the message formats and communications required for a particular type of endpoint class. If an application interfaces to two different endpoint classes, such as a remote banking endpoint device and a host, then two Endpoint Class Handler objects are required—a Voice Response Unit Endpoint Class Handler object for communications with the remote banking endpoint device and a Host Endpoint Class Handler object for communications with the host.

This section includes the following topics:

- An overview of how the Endpoint Class Handler object fits into message processing
- A discussion of the normal processing performed by the Endpoint Class Handler object
- A discussion of the configuration requirements for the Endpoint Class Handler object
- A list of the member functions supported by the Endpoint Class Handler object

Note: This section provides an overview of all possible Endpoint Class Handler object functionality. An endpoint-specific Endpoint Class Handler object (for example, Voice Response Unit Endpoint Class Handler object) will only perform

the functions necessary for a given endpoint. The processing performed for an endpoint is dictated by the specific processing requirements for that endpoint. The Voice Response Unit Endpoint Class Handler object provides the facilities to process messages received from an Remote Banking Standard Interface ISO 8383, 1993 message format.

Overview

The Endpoint Class Handler object is typically called by the XPNET Interface object. The Endpoint Class Handler object determines the message class for a message and whether context must be retrieved when routing the message. It can also determine a single symbolic destination for the message if it is constructed to do so. For BASE24, the message class associated with a message is typically composed from the product ID, message type, and message variant. Any combination of message fields, however, can be used to derive the message class.

An Endpoint Class Handler object uses a combination of fields from the message to determine how to call the Endpoint Class Handler Services object. The exact fields used can vary from one Endpoint Class Handler object to another. For example, the Voice Response Unit Endpoint Class Handler object uses the message type and the function code from the external message to determine how the Endpoint Class Handler Services object is called. If the message type or function code indicates that the message is either a continuation of a multiple account select request or a network management message acknowledging a key change, the Voice Response Unit Endpoint Class Handler object requests that context be retrieved prior to routing the message. If the Endpoint Class Handler object determines that context does not exist for the message, the Endpoint Class Handler Services object is called to route the message only.

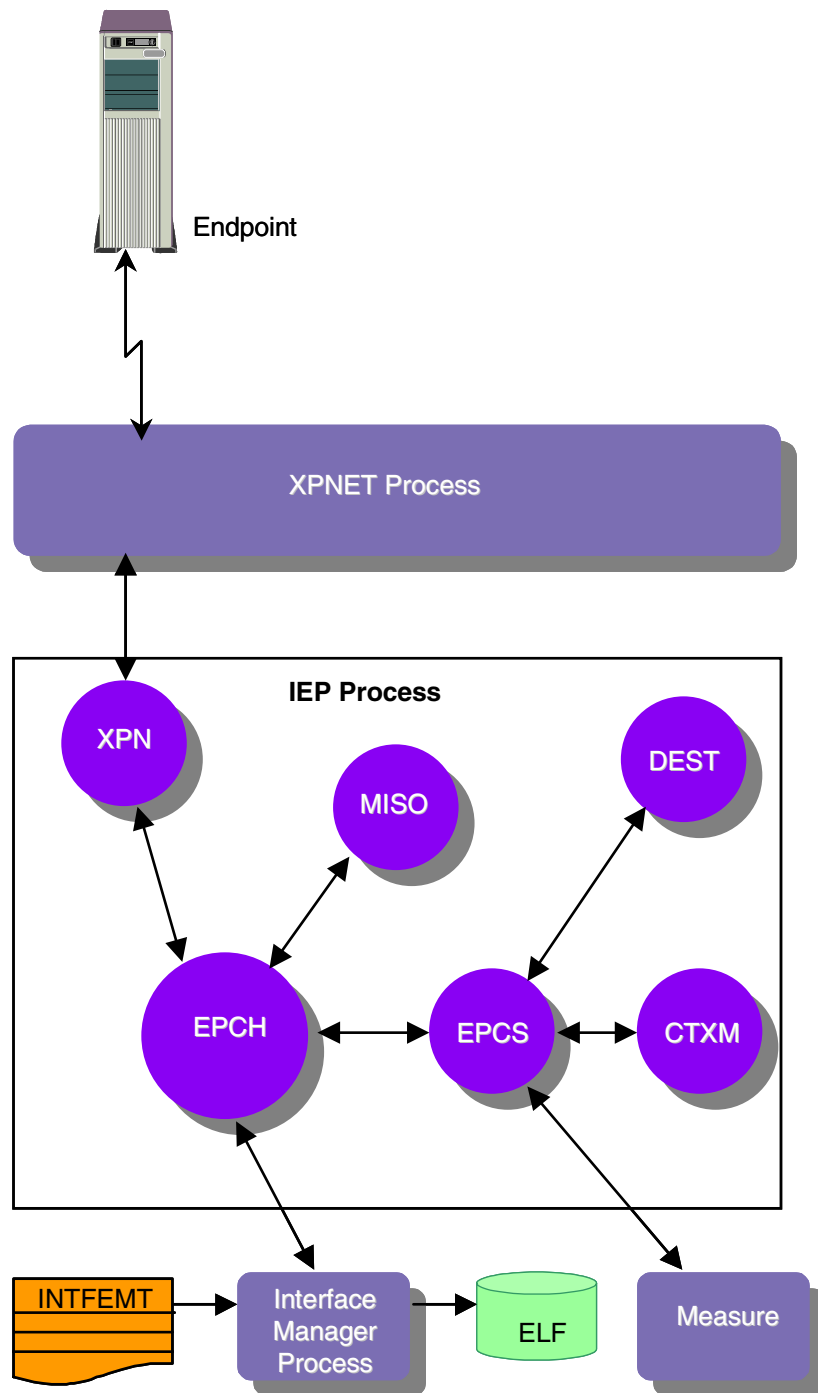
The Endpoint Class Handler object can perform the following functions:

- Measure counter support
- Message identification
- Message class generation (deriving the message class from messages received from an endpoint)
- Message expansion (using the Message ISO object)
- Message-based routing (using the Endpoint Class Handler Services and Destination objects)
- Network overload management processing
- Context storage and retrieval (using the Endpoint Class Handler Services and Context Manager objects)
- Symbolic destination determination (if the object is constructed to do so) and routing (using the Endpoint Class Handler Services object)
- Exception reporting (using the Interface Manager process)

The functions of the Endpoint Class Handler object are depicted in the illustration on the following page. A key to the abbreviations is provided below.

Key

CTXM	=	Context Manager object
DEST	=	Destination object
ELF	=	Exceptions Log File
EPCH	=	Endpoint Class Handler object
EPCS	=	Endpoint Class Handler Services object
IEP	=	Integrated Endpoint
INTFEMT	=	Interface extended memory table
MISO	=	Message ISO object
XPNET	=	XPNET Interface object



Initialization

During phase one of initialization, all Endpoint Class Handler objects obtain frame space and initialize their internal state with the Application Network Services object.

During phase two of initialization, Endpoint Class Handler objects call the Endpoint Class Handler Services object with the Register member function in preparation for context processing. This member function specifies the context requirements for the Endpoint Class Handler object. When received, the Endpoint Class Handler Services object registers these requirements with the Context Manager object using another Register member function.

Depending on the specific endpoint processing and application requirements, the Endpoint Class Handler object may also register with application processes. For example, in BASE24-telebanking, the Endpoint Class Handler object registers with the Interface Manager process.

Normal Processing

This subsection discusses the normal processing performed by the Endpoint Class Handler object.

Message Reception

The Endpoint Class Handler object receives the Message In member function from the XPNET Interface object when either a message is received from the endpoint or a station status message is received. For messages received other than station status messages, the object performs message conversion and routing as described below and on the following pages. Station status messages are forwarded to the Interface Master process.

Message Conversion

Every Endpoint Class Handler object is written to handle one of the following message types:

- Fixed structure messages with known offsets and field lengths
- International Organization for Standardization (ISO) 8583 bit-mapped messages
- Variable length messages that use field separator (FS) characters to delimit fields

Fixed Structure Messages

Fixed structure messages are the simplest types of messages to convert by the Endpoint Class Handler object because the location of each field in the message is known. The Endpoint Class Handler object does not have to search through the entire message or bit map to determine whether a field exists in the message. With fixed structure messages, the message is directly overlaid with the correct internal message structure. The internal message structure is known by the application processes (for example, Integrated Authorization Server processes) that process messages received from an endpoint. In this case, the Endpoint Class Handler object accesses the fields directly from the external message.

ISO 8583 Bit-Mapped Messages

For International Organization for Standardization (ISO)-based endpoints, the bit map from the message maps the ISO message into a fixed Internal Transaction Data (ITD) format. For efficient processing, the message is converted to a known fixed format before the message is processed by an application. Otherwise, the application must search the ISO bit map each time it needs a particular field. By having the message in a fixed format, the application knows where each field is in the message and can directly access a field each time a particular field is needed for processing.

The Endpoint Class Handler object calls the Message ISO object to translate ISO-based messages into Internal Transaction Data (ITD) formatted messages. This is known as message expansion. The Message ISO object is also called to map ITD-formatted messages back into bit-mapped ISO-formatted messages. This is known as message compression. Typically, message compression is performed only by Integrated Authorization Server processes and not by the Integrated Endpoint process.

Field Separated Messages

Variable length field-separated messages are the most difficult to convert, since fields in the message can be provided in any order. Each field is preceded by a field identifier that identifies each field in the message. The field identifiers must be known by the Endpoint Class Handler object for it to be able to extract the desired fields from the message to build the Internal Transaction Data (ITD) message. As with ISO messages, it is more efficient to translate the field-separated message into a fixed ITD structure once, rather than doing this each time a field is needed for processing.

Message Routing

When an Endpoint Class Handler object receives a message from an external entity, the message is examined to determine the message type. It can also determine a single symbolic destination for the message if it is constructed to do so.

The Endpoint Class Handler object is responsible for determining the message type and message class. The message class is typically based on the product ID, message type identifier, and message variant from the message, although any combination of message fields may be used. If the message type indicates that context is already stored, the Endpoint Class Handler object formats a context key

to retrieve the context. The Endpoint Class Handler object then formats the Message Route Context Retrieve member function and calls the Endpoint Class Handler Services object to route the message and retrieve context as needed. The Message Route member function is used if no previously stored context exists for the message. Both of these member functions specify the source class and message class associated with the message to determine the destinations for the message using message-based routing.

If the Endpoint Class Handler object is constructed to determine a single symbolic destination for a message based solely on the message contents, and a symbolic destination is determined, the object calls the Endpoint Class Handler Services object with the Message Route Symbolic member function to route the message. This function can also contain the source class and message class if the message is to be routed to additional destinations determined through message-based routing.

For messages originating from an Integrated Authorization Server process that are to be sent to an external endpoint, the Endpoint Class Handler is not used. Messages are sent directly to the Endpoint Class Handler Services object. For all messages received from Integrated Authorization Server processes, the process sending the message indicates whether context needs to be stored for the message and whether message-based routing is required.

Message Failure

The Endpoint Class Handler object receives an External Message Failed member function or an External Message Failed with Context member function from the Endpoint Class Handler Services object when a message cannot be delivered by the XPNET process. The request received indicates whether context has been stored for the message. If context has been stored, the Endpoint Class Handler object builds a context key. Then, based on whether context exists for the failed message, the Endpoint Class Handler object calls the Endpoint Class Handler Services object with either a Message Route member function or a Message Route Context Retrieve member function. The Endpoint Class Handler Services object then retrieves context if indicated and routes the failed message.

Device-Specific Functions

The Endpoint Class Handler object handles all the device-specific functions for the particular endpoint it services. In this respect, the Endpoint Class Handler object functions much like a device handler or interface process in a procedural environment. For example, the Voice Response Unit Endpoint Class Handler object used with the BASE24-telebanking product works in conjunction with the

Interface Manager process to maintain information on remote banking endpoint device stations. It checks the station type of all messages received from the endpoint. Along with the XPNET process, the Endpoint Class Handler object also maintains the communications link to the endpoint it communicates with.

Network Overload Management

Note: The following is true only if your endpoint-specific Endpoint Class Handler object is constructed to work this way. This is the way ACI has coded the Voice Response Unit Endpoint Class Handler object for the BASE24-telebanking application. There are many different possible actions that can be taken when network overload is detected for an endpoint process or station. For example, you may want your application to never discard any messages received from an endpoint. On the other hand, you may want your application to discard all new transaction request messages when network overload is detected.

When the Voice Response Unit Endpoint Class Handler object receives a transaction request message, it checks the Network Overload Management Percent field in the network message header to determine whether network overload has been detected. A value greater than 100 percent indicates that overload has been detected. If so, the Voice Response Unit Endpoint Class Handler begins discarding new transaction request messages that it receives. Response and advice messages are not discarded. Multiple leg transaction requests (for example, multiple account select transactions) are also never discarded when network overload is detected. This allows transactions in progress to complete, but prevents new transactions from entering the system. As transactions complete, fewer messages will be queued and the queueing situation will be alleviated. Once the network overload situation is alleviated, the XPNET Interface object will indicate the number of messages that have been discarded each time the object evaluates the queue. This is done in accordance with the interval specified in the Queue Evaluation Delay (QED) Logical Network Configuration File (LCONF) param.

If the queue continues to build despite these actions, the number of messages in the Integrated Endpoint process queue will eventually cause the XPNET process to either save messages to an extended memory table or discard them before they are delivered to the Integrated Endpoint process.

Optional Processing

The following processing can be optionally provided if required by the endpoint being processed. This processing is provided for BASE24-telebanking.

EMT Notify Member Functions

The Endpoint Class Handler object receives the EMT Notify member function from the Interface Manager object when the Interface Manager object has reallocated a new Interface extended memory table. Upon receipt of the EMT Notify member function, the Endpoint Class Handler object moves the object address, station map address, station table length, and the Interface extended memory table name sent by the Interface Manager object into its own frame.

Exception Handling

Any interface endpoint exceptions encountered by the Endpoint Class Handler object are sent to the Interface Manager object in the Interface Master process to be recorded in the Exceptions Log File (ELF).

Extended Memory Table Usage

The Endpoint Class Handler object uses the Interface extended memory table to retrieve the Measure counter name and the message class wildcard options for an interface instance if the Integrated Endpoint process is developed for a BASE24 endpoint. This information may be obtained from another source if the Integrated Endpoint process is developed for a non-BASE24 application.

For ISO-based messages, the object also accesses the External Transaction Data Table in the Interface extended memory table to retrieve the bit map needed to format administrative messages, such as a reject message.

Generic Endpoint Class Handler Object

All Endpoint Class Handler objects are built using the Generic Endpoint Class Handler object as a template. This object includes standard procedures that are executed by all Endpoint Class Handler objects as well as skeleton procedures that must be developed specifically on behalf of a particular endpoint.

Configuration

This subsection lists the configuration requirements for the Endpoint Class Handler object.

Kernel Configuration

At least one Endpoint Class Handler object must be configured to every Integrated Endpoint process running in the ITS environment.

A record must be added to the Object File (OBJ) for each Endpoint Class Handler object. Once this OBJ record has been established, a record associated with the object must be added to the Process/Object Relationship File (PORF). For detailed instructions on configuring the kernel environment, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

Additional records may need to be configured in the EMT/Object Relationship File (EORF) and the Assign/Object Relationship File (AORF), depending on whether the endpoint-specific Endpoint Class Handler object uses any extended memory tables or assigns that can be reallocated or reread using the EMT WARMBOOT or ASSIGN WARMBOOT commands, respectively.

LCONF Assigns and Params

The Generic Endpoint Class Handler object currently does not use any Logical Network Configuration File (LCONF) assigns or params in its processing. Endpoint-specific Endpoint Class Handler objects may use some assigns or params, depending upon the type of endpoint processing required. Any endpoint-specific LCONF assigns or params used by an Endpoint Class Handler object for BASE24 products are described in the ***BASE24 Logical Network Configuration File Manual***.

Member Functions

This subsection documents the member functions sent and received by the Endpoint Class Handler object. Integrated Endpoint member functions are described in section 1. Kernel member functions are described in the ***BASE24 ITS Kernel Objects Reference Manual***. Core member functions are described in the application-specific transaction processing manual.

Note: Many of the member functions presented here for the Endpoint Class Handler object are optional based on the specific processing requirements of the endpoint being serviced. For example, Message ISO lexicon functions are only required if the Endpoint Class Handler object is processing messages received from an ISO endpoint.

Member Functions Sent

The table below shows the member functions sent by the Endpoint Class Handler object, arranged according to the lexicon of which they are a part.

Lexicon Name	Member Function
All lexicons	Version
Endpoint Class Handler Object to Endpoint Class Handler Services Object	Message Route
	Message Route Context Retrieve
	Message Route Symbolic
	Register
Object to Application Network Services Object	Register
Object to Event	Log
Object to Interface Manager	ELF Write [*]
	Register
Object to Message ISO Object	Message Compress [†]
	Message Expand [†]

Lexicon Name	Member Function
Object to Process Control	ISCF Write
Object to Space	Memory Get
Object to Transport Layer Object	Message Out
Object to Trace	Add Trace

* This member function is used only if an Exceptions Log File (ELF) is used.

† These member functions are used only if the Endpoint Class Handler object services an ISO endpoint and some of these member functions are used only in core processes rather than in the Integrated Endpoint process.

Member Functions Received

The table below shows the member functions received by the Endpoint Class Handler object, arranged according to the lexicon of which they are a part.

Lexicon Name	Member Function
All lexicons	Version
Application Network Services Object to Object	Initialize
	Notify
	Stop ANS
	System Message
Message ISO Object to Object	Message Compress Unique*
	Message Expand Unique*
	Message Trace Unique

Lexicon Name	Member Function
Process Control to Object	Command Message
	EMT Warmboot [†]
	File Close/Open
	LCONF Warmboot
	Param Warmboot
TLO to Object	File I/O Completion
	Message Failed
	Message In
	System Time Signal
Trace to Object	Start Trace
	Stop Trace

* These member functions are only used if the Endpoint Class Handler object services an ISO endpoint and some of these member functions are only used in core processes rather than in the Integrated Endpoint process.

[†] This member function is only used if the Endpoint Class Handler object supports an extended memory table that can be warmbooted.

Section 5

Endpoint Class Handler Services Object

The Endpoint Class Handler Services object provides common services required by endpoint-specific objects, such as all Endpoint Class Handler objects in Integrated Endpoint processes and endpoint-specific core objects in Integrated Authorization Server processes. These services include Measure counter manipulation, message-based routing (using the Destination object), timer management, context storage and retrieval (using the Context Manager object), and external message formatting. This section describes the Endpoint Class Handler Services object and includes the following topics:

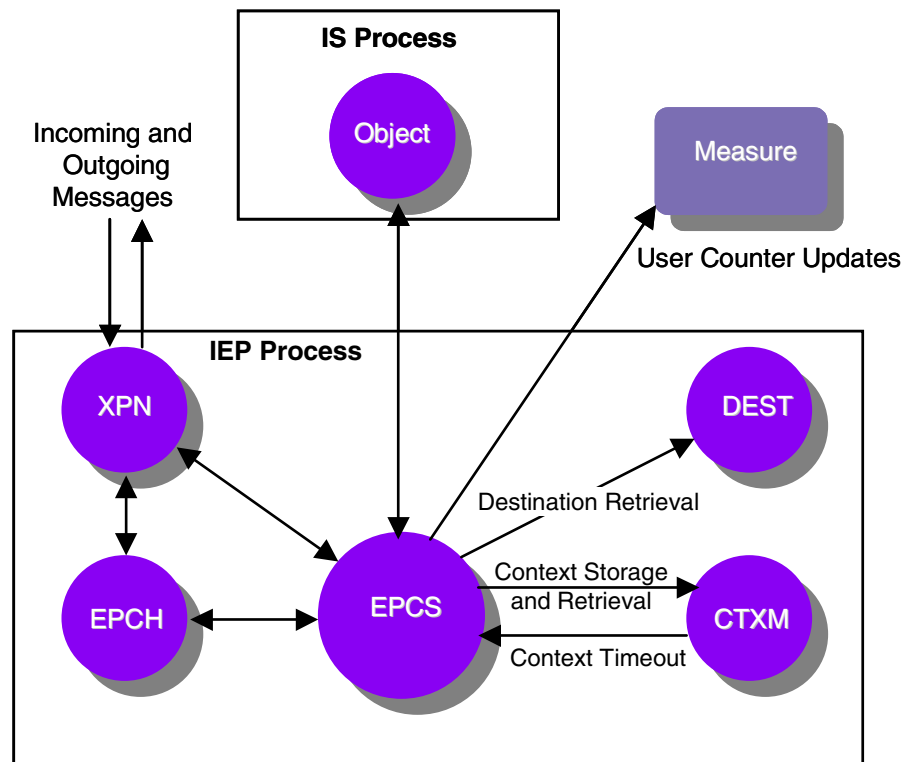
- An overview of how the Endpoint Class Handler Services object fits into message processing
- A discussion of the normal processing performed by the Endpoint Class Handler object
- A discussion of the configuration requirements for the Endpoint Class Handler Services object
- A list of member functions supported by the Endpoint Class Handler Services object

Overview

The Endpoint Class Handler Services object provides all the common services required by endpoint-specific Endpoint Class Handler objects in the Integrated Endpoint process or endpoint-specific core objects in external Integrated Server processes.

The rationale for having an Endpoint Class Handler services object is that various types of endpoint-specific objects all have some similar processing requirements. The Endpoint Class Handler Services object provides these common services.

The functions of the Endpoint Class Handler Services object are depicted in the illustration on the following page. A key to the abbreviations follows the illustration. Detailed processing descriptions are provided later in the section.



Key

- CTXM = Context Manager object
- DEST = Destination object
- EPCH = Endpoint Class Handler object
- EPCS = Endpoint Class Handler Services object
- IEP = Integrated Endpoint
- IS = Integrated Server process
- XPIN = XPNET Interface object

Normal Processing

This subsection describes the normal processing performed by the Endpoint Class Handler Services object.

External Message Routing

The Endpoint Class Handler Services object is called by the Endpoint Class Handler object to route each external message it receives. It can also be called by an object in an Integrated Server process (for example, an Integrated Authorization Server process) that has an external message to be routed.

Endpoint Class Handler Object Messages

When the Endpoint Class Handler Services object receives the Message Route member function, it performs message-based routing by calling the Destination object to retrieve the message destinations, and formats an external message for each destination. No context is stored for the message.

When the Endpoint Class Handler Services object receives the Message Route Context Retrieve member function, it performs message-based routing by calling the Destination object to retrieve the message destinations, retrieves context by calling the Context Manager object, and formats an external message for each destination.

When the Endpoint Class Handler Services object receives the Message Route Symbolic member function, it formats an external message for the specified symbolic destination and performs message-based routing by calling the Destination object to retrieve additional message destinations. The Endpoint Class Handler Services object also formats an external message for each additional destination determined during message-based routing. No context is stored for the message.

Core Object Messages

When the Endpoint Class Handler Services object receives the Message member function from an application (core) object, it performs message-based routing by calling the Destination object, and formats external messages for each destination found. No context is stored for this function.

When the Endpoint Class Handler Services object receives the Message Context Store member function from an application object, it performs message-based routing by calling the Destination object and store context by calling the Context Manager object. An external message is formatted for each destination found.

Destination Retrieval

When message-based routing is indicated in a Message Route, Message Route Context Retrieve, Message Route Symbolic, Message, or Message Context Store member function, the Endpoint Class Handler Services object calls the Destination object with the Destination Retrieve member function. For each destination retrieved by the Destination object, the Endpoint Class Handler Services object formats an appropriate external message.

The object obtains the message class wildcard flag value from one of the above member functions received and includes it in the Destination Retrieve member function. The message class wildcard flag is used by the Destination object to determine how wildcard values are used for the message class when searching the Destination extended memory table.

Message-based routing is only performed if the member function specifies the source class, message class, and message class wildcard flag. The Message Route Symbolic, Message, and Message Context Store member functions can specify a single symbolic destination in the request and may not require message-based routing. In such cases, the message-based routing parameters are not included in the request.

Context Storage and Retrieval

When the Endpoint Class Handler Services object receives the Message Context Store member function, the object calls the Context Manager object with the Context Store member function to store message context.

When the Endpoint Class Handler Services object receives the Message Route Context Retrieve function, the object calls the Context Manager object with the Context Retrieve member function to retrieve previously stored message context. The Message Route Context Retrieve member function includes the context key.

External Message Formatting

For each destination to which a message is to be routed, the Endpoint Class Handler Services object formats an external message. The type of external message formatted depends on whether the message is a failed message and whether stored context exists for the message. The following table shows the type of member function used to format the message based on this criteria.

Message Failed	Stored Context	Member Function
Yes	No	External Message Failed
Yes	Yes	External Message Failed with Context
No	No	External Message
No	Yes	External Message with Context

External messages are enveloped within the Message Out member function which is sent to the XPNET Interface object. The XPNET Interface object sends each message to the XPNET process for delivery.

Timed Out Message Processing

If the timer for stored context expires before the context is retrieved, the object receives the Context Timeout member function from the Context Manager object. The Endpoint Class Handler Services object then calls the Context Manager with the Context Retrieve member function. The retrieved context is included in an External Message Timeout member function that is sent to the message originator (for example, an Integrated Authorization Server process).

If an external message is received for which the context timer has already expired (for example, a late message), the object formats an External Message Late member function that is sent to the message originator.

External Message Sending

All external messages formatted by the Endpoint Class Handler Services object are sent to the XPNET Interface object enveloped within the Message Out member function. The XPNET Interface object then sends the external message to the XPNET process for delivery to the specified destination.

Measure Counter Manipulation

Note: The following paragraphs document the way Measure counters are implemented in support of BASE24. If your Integrated Endpoint process supports applications other than BASE24, Measure counters may be implemented differently from that shown here.

Measure counters are maintained for all financial responses, financial advice and reversal responses, timed out messages, and failed messages processed by the Integrated Endpoint process. The Endpoint Class Handler object or endpoint-specific core object provides the Measure user counter name for appropriate messages in Message Route, Message Route with Context, Message Route Symbolic, Message, or Message Context Store member functions sent to the Endpoint Class Handler Services object.

For the BASE24-telebanking product, Measure counter names are optionally established by interface instance in the ITS Interface Configuration File (IFCF). If a Measure counter name is not configured for an interface instance in the IFCF, then processing statistics are not maintained for that interface instance. Each interface instance is associated with a group of stations. The information in this file is used to build the Interface extended memory table. The Endpoint Class Handler object reads this table to obtain the Measure user counter name associated with each message received from a station and includes it in Message, Message Route Context Retrieve, and Message Route Symbolic member functions sent to the Endpoint Class Handler Services object.

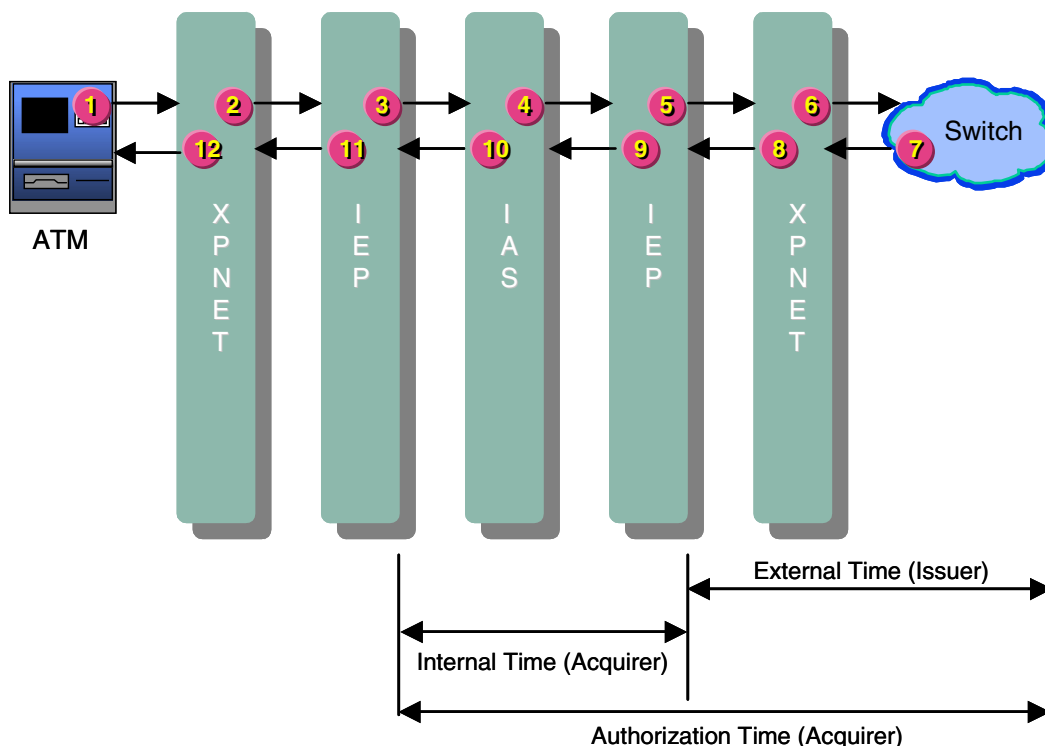
Each time a Measure counter is updated, the Endpoint Class Handler Services object passes several parameters to the Tandem Measure product. Count updates are always increased by one. The following information is passed to Measure to update a counter:

- User counter offset (this is returned when a MEASUREBUMPINIT operation is performed)
- Update (bump) type
- Amount to be updated (for example, a time duration)
- User counter index

The Endpoint Class Handler Services object is responsible for tracking the internal and external time for a transaction. The internal time is the amount of time required by integrated transaction services (ITS) components to process a transaction once it has been received by an Integrated Endpoint process. The

external time is the time required to send and receive messages from an issuer endpoint to the Integrated Endpoint process. The authorization time is defined as the total of both the internal and external times.

Internal, external, and authorization times are depicted in the following illustration. A key to the abbreviations follows the illustration.



Key

ATM = Automatic Teller Machine
IAS = Integrated Authorization Server process
IEP = Integrated Endpoint process

Each Measure user counter contains a 15-field array of position counters. The message class determines which position counters in the Measure user counter are updated for each message. The message class is made up of three parts: a two-character product indicator, a four-character message type, and a two-character message variant. The message type and message variant are the two parts of the message class used to determine which counters are updated.

Message Variant

The message variant indicates whether a message was delivered normally, failed, was timed out, was returned, was rejected, or was returned late.

If the message variant indicates a failed message, the Endpoint Class Handler Services object immediately updates the position counter for failed messages. If the message variant is either normal or timed out, the object determines the message type. If the message variant is none of the types mentioned above, the message will not cause any Measure position counter updates.

Message Type

For messages with a message variant indicating a normal message, the Endpoint Class Handler Services object determines whether the message is one of the following International Organization for Standardization (ISO) message types:

- 1110 or 1210 Financial response
- 1130 or 1230 Advice response
- 1430 Reversal Advice response

If the message is not one of these message types, the position counters are not updated.

For messages with a message variant indicating that the message timed out, the message is considered to fall into one of two possible groups. One group consists of financial transaction requests and responses. The other group consists of advice requests, advice responses, reversal requests, and reversal responses. The following ISO message types are possible for a message that timed out:

- 1100 and 1101 Financial request
- 1110 Financial response
- 1120 and 1121 Advice request
- 1200 and 1201 Financial request
- 1210 Financial response
- 1220 and 1221 Advice request
- 1230 Advice response

- 1420 and 1421 Reversal Advice request
- 1430 Reversal Advice response

If the message type is not one of these message types, the position counters are not updated.

Position Counter

Once the message type has been determined, the Endpoint Class Handler Services object uses the message class passed as a parameter and sets an external or internal flag to determine the position counter as defined in the following table.

The position counter identifies the position of that counter in a 15-field array defined for each Measure user counter name. A user counter name must be established for each entity (for example, interface instance) for which statistics are to be collected by the Integrated Endpoint process for your application.

For detailed information on the configuration and reporting of Measure user counters, refer to the ***BASE24 IEP Configuration Manual***.

Counter	Position
Late response count	0
Late response time	1
Failed message count	2
For financial transaction requests and responses:	
External authorization count	3
External authorization time	4
Internal authorization count	5
Internal authorization time	6
Authorization count	7
Authorization time	8
Timeout count	9

Counter	Position
For financial advice and reversal advice requests and responses:	
External authorization count	10
External authorization time	11
Internal authorization count	12
Internal authorization time	13
Timeout count	14

When updating position counters for financial responses, the internal authorization count, internal authorization time, authorization count, and authorization time counters are updated at the same time. The internal counters and the authorization counters are always updated together.

The following logic determines which position counters are updated:

- For a message class indicating a failed message, position counter 2 is updated.
- For a message class indicating a normal financial response (rather than a timed out or failed message) and a flag indicating that the response is external, position counters 3 and 4 are updated. If the flag indicates that the response is internal, position counters 5 through 8 are updated.
- For a message class indicating that a financial transaction request or response has timed out, position counter 9 is updated.
- For financial and reversal advice responses, position counters 10 and 11 are updated if a flag indicates that the response is external. If a flag indicates that the response is internal, position counters 12 and 13 are updated.
- For a timeout of a reversal of a financial advice request or response, position counter 14 is updated.

Message Routing Timer Processing

The following timer processing is performed when a message is received by the Endpoint Class Handler Services object.

If a message received is not a failed message, the Endpoint Class Handler Services object calculates the message queue time as described below. If the message queue time is 0, then no queue time limit is imposed.

The authorization and internal times needed to process a message are calculated for each message processed. These times are recorded in Measure position counters.

For messages received from an external application (in the Message member function), the Endpoint Class Handler Services object calculates the message queue time. If the maximum queue time field in the Message member function is 0, then the message queue time is also set to 0. If the maximum queue time field is not 0, then the message queue time is calculated. It is set to the minimum of either one of the following:

- The value in the maximum queue time field.
- The value of the delivery timestamp field in the Message member function minus the current Julian timestamp. If the Julian timestamp is greater than or equal to the delivery timestamp, the object formats an External Message Failed member function, envelopes it in a Message Out member function, and sends it to the appropriate external entity. No further processing is performed for the Message member function.

The Endpoint Class Handler Services object receives a Context Timeout member function from the Context Manager object when a timer associated with stored message context expires. The context itself is included in the Context Timeout member function.

When the context is retrieved, the Endpoint Class Handler Services object sets the response type to indicate an external message timeout, updates the appropriate Measure position counters, formats an External Message Timeout member function, envelopes this function in a Message Out member function, and sends it to the appropriate external entity.

Configuration

This subsection lists the configuration requirements for the Endpoint Class Handler Services object.

Kernel Configuration

The Endpoint Class Handler Services object must be configured to every Integrated Endpoint process running in the ITS environment.

A record must be added to the Object File (OBJ) for the Endpoint Class Handler Services object. Once this OBJ record has been established, a record associated with the object must be added to the Process/Object Relationship File (PORF). For detailed instructions on configuring the kernel environment, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

LCONF Assigns and Params

The Endpoint Class Handler Services object currently does not use any Logical Network Configuration File (LCONF) assigns or params in its processing.

Member Functions

This subsection lists the member functions sent and received by the Endpoint Class Handler Services object. Integrated Endpoint member functions are described in section 1. Kernel member functions are described in the ***BASE24 ITS Kernel Objects Reference Manual***.

Member Functions Sent

The table below shows the member functions sent by the Endpoint Class Handler Services object, arranged according to the lexicon of which they are a part.

Lexicon Name	Member Function
All lexicons	Version
Endpoint Class Handler Services Object to Object	External Message
	External Message with Context
	External Message Failed
	External Message Failed with Context
	External Message Late Response
	External Message Timeout
Object to Application Network Services Object	Register
Object to Context Manager	Context Retrieve
	Context Store
	Register
Object to Destination	Destination Retrieve
Object to Event	Log
Object to Space	Memory Get

Lexicon Name	Member Function
Object to Trace	Add Trace
Object to Transport Layer	Message Out

Member Functions Received

The table below shows the member functions received by the Endpoint Class Handler Services object, arranged according to the lexicon of which they are a part.

Lexicon Name	Member Function
All lexicons	Version
Application Network Services Object to Object	Initialize
Application to Object	Message
Context Manager to Object	Context Timeout
Endpoint Class Handler Object to Endpoint Class Handler Services Object	Message Route
	Message Route Context
	Register
Object to Endpoint Class Handler Services Object	Message
	Message Context Store
Transport Layer Object to Object	Message In
	Message Failed
Trace to Object	Start Trace
	Stop Trace

ACI Worldwide Inc.

Section 6

Message ISO Object

The Message ISO object provides message expansion and compression of International Organization for Standardization (ISO) messages to or from any specified fixed structure, such as the Internal Transaction Data (ITD) format. This object is only required for Integrated Endpoint processes that service ISO endpoints. This section describes the Message ISO object and includes the following topics:

- An overview of how the Message ISO object fits into message processing
- A discussion of the normal processing performed by the Message ISO object
- A discussion of the configuration requirements for the Message ISO object
- A list of the member functions supported by the Message ISO object

This object is only required if the Integrated Endpoint process handles ISO-based messages from the endpoint class.

Overview

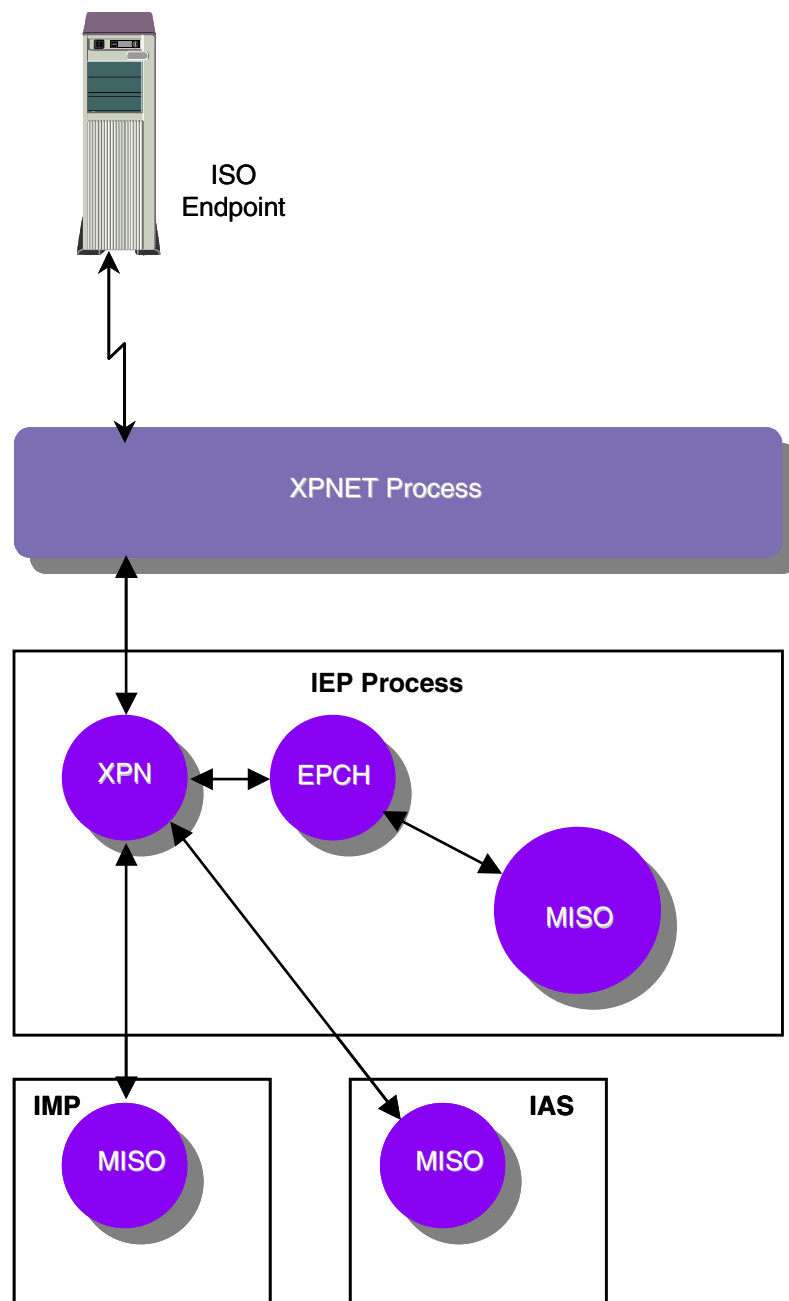
The Message ISO object expands and compresses International Organization for Standardization (ISO) messages to or from any specified structure, such as the Internal Transaction Data (ITD) format. The ISO message format used is defined by the client object. Any ISO-based Endpoint Class Handler object can communicate with the Message ISO object to expand a message. The message can be authenticated using a message authentication code (MAC) during message expansion. A personal identification number (PIN) can be translated during message compression. For BASE24, message expansion is the only function performed by the Message ISO object residing within the Integrated Endpoint process. The following functions can be performed by the Message ISO object residing in Interface Master processes and Integrated Authorization Server processes:

- Message compression
- MAC generation, translation, and verification
- PIN translation before the PIN is placed in the external message

ISO messages processed by the Message ISO object must adhere to the standards published in the 1993 ISO 8583 standard, *Financial transaction card originated messages—Interchange message specifications*.

Note: The client object calling the Message ISO object is responsible for providing the appropriate PIN keys for PIN translation.

The functions of the Message ISO object are depicted in the illustration on the following page. A key to the abbreviations follows the illustration.

**Key**

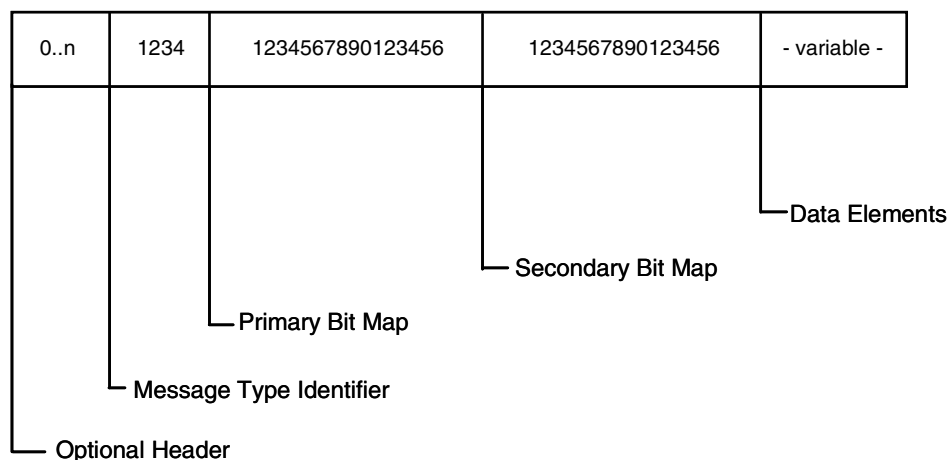
- EPCH = Endpoint Class Handler object
- IAS = Integrated Authorization Server process
- IEP = Integrated Endpoint
- IMP = Interface Master process
- ISO = International Organization for Standardization
- MISO = Message ISO object
- XPN = XPNET Interface object

Normal Processing

This subsection describes the message formats used by the Message ISO object as well as the processing performed by the object.

ISO External Message Format

The following illustration depicts the external message format of ISO messages processed by the Message ISO object. Each part of the message format is described following the illustration.



Optional Header — Contains application-specific information and is not used by the Message ISO object. The header is optional on ISO messages.

Message Type — A four-digit code identifying the general function of the message. The message type is required in all messages and is parsed by the application.

Primary Bit Map — Represents 64 bits of data used to identify the presence or absence of the first 64 data elements in the message. If binary-coded decimal notation (BCD) fields are transmitted as BCD, the primary bit map is sent as 64 bits (8 character positions). If BCD fields are transmitted as American National Standard Code for Information Interchange (ASCII), the primary bit map is converted from or to 16 bytes of display data using hexadecimal equivalents. Each group of four bits is assigned a hexadecimal equivalent (0–9, A–F).

Secondary Bit Map — Represents 64 bits of data used to identify the presence or absence of data elements 65–128 in the message. The first bit of the primary bit map indicates the presence or absence of the Secondary Bit Map (P-1). If BCD fields are transmitted as BCD, the secondary bit map is sent as 64 bits (8 character positions). If BCD fields are transmitted as ASCII, the secondary bit map is converted from or to 16 bytes of display data using hexadecimal equivalents. Each group of four bits is assigned a hexadecimal equivalent (0–9, A–F).

Data Elements — Individual fields containing application data. Each bit of the primary bit map and secondary bit map indicates the presence or absence of a particular data element. Each data element is identified as a fixed or variable length type field. If the data element is fixed, the field length is defined by ISO. If the data element is variable, the field length is contained in the external message prior to the actual field, as defined by ISO. Each data element also defines whether the field is transmitted as BCD, ASCII, or extended binary-coded decimal interchange code (EBCDIC).

Message Compression

The following functionality is only performed by Message ISO objects running in Interface Master processes or Integrated Authorization Server processes. It is not currently performed for BASE24 by the Message ISO object running in the Integrated Endpoint process.

When a request, advice, or reversal transaction message is sent to an ISO issuer, or when a response or acknowledgment is sent to an ISO acquirer, the Message ISO object in an Integrated Authorization Server process translates the fixed Internal Transaction Data (ITD)-formatted message into an external bit-mapped ISO-message format, recognizable by the endpoint. This is referred to as message compression. When a message is compressed, the object may also authenticate the message using a message authentication code (MAC) or translate a PIN. Compression starts with the primary bit map and continues until the last bit indicated in the primary or secondary bit map is processed. The client object is responsible for building the message header, if one is used, and for providing the message type.

Message Expansion

When a request, advice, or reversal transaction message is received from an ISO acquirer, or when a response or acknowledgment is received from an ISO issuer, the Message ISO object translates the message from external bit-mapped ISO message format into the fixed Internal Transaction Data (ITD) format,

recognizable by ITS objects. This is referred to as message expansion. When a message is expanded, the Message ISO object can also authenticate the message using a message authentication code (MAC). The client object is responsible for translating the message header, if one is sent, and for handling the message type.

Configuration

This subsection lists the configuration requirements for the Message ISO object.

Kernel Configuration

The Message ISO object must be configured to every Integrated Endpoint process running in the ITS environment that services an ISO endpoint. The object may also need to be configured to Interface Master and Integrated Authorization Server processes.

A record must be added to the Object File (OBJ) for the Message ISO object. Once this OBJ record has been established, records associated with the object must be added to the Process/Object Relationship File (PORF). For detailed instructions on configuring the kernel environment, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

LCONF Assigns and Params

The Message ISO object currently does not use any Logical Network Configuration File (LCONF) assigns or params in its processing.

Member Functions

This subsection documents the member functions sent and received by the Message ISO object. Integrated Endpoint member functions are described in section 1. Kernel member functions are described in the ***BASE24 ITS Kernel Objects Reference Manual***.

Member Functions Sent

The table below shows the member functions sent by the Message ISO object, arranged according to the lexicon of which they are a part.

Lexicon Name	Member Function
All lexicons	Version
Message ISO to Object	MAC Generate/PIN Translate
	MAC Verify
	Message Compress Unique
	Message Expand Unique
	Message Trace Unique
Object to ANS	Register
Object to Event	Log
Object to Space	Memory Get
Object to Trace	Add Trace

Member Functions Received

The table below shows the member functions received by the Message ISO object, arranged according to the lexicon of which they are a part.

Lexicon Name	Member Function
All lexicons	Version
ANS to Object	Initialize
Object to Message ISO	Message Compress
	Message Expand
Trace to Object	Start Trace
	Stop Trace

ACI Worldwide Inc.

Section 7

Integrated Endpoint Process Transaction Processing

This section provides in-depth descriptions of Integrated Endpoint process object processing for various transaction scenarios. It is provided as an aid in understanding the processing performed by Integrated Endpoint process objects. Transaction flow descriptions are provided for the following scenarios:

- Transaction request authorized by an Integrated Authorization Server process
- Transaction response generated by an Integrated Authorization Server process
- Financial completion sent to an Integrated Authorization Server process
- Financial transaction authorized on a host
- Context timeout
- Network overload detected

Note: Only process-level interaction is shown in the transaction flow illustrations in this section. Object interactions are not shown, although all relevant object processing is provided in the processing descriptions accompanying each illustration.

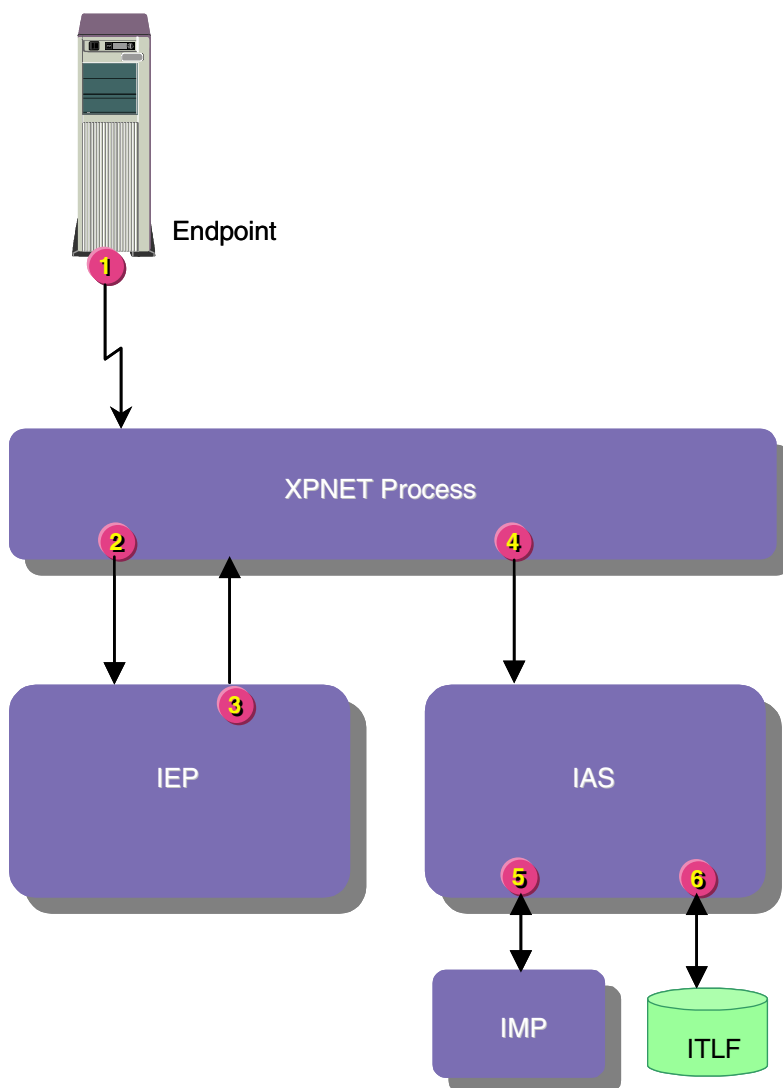
Transaction Request Authorized by an Integrated Authorization Server Process

The following transaction flow depicts the flow of information through the Integrated Endpoint process when a transaction request message is received from an acquirer endpoint and is authorized by an Integrated Authorization Server process. This is an example of simple routing. Only the request portion of the transaction is depicted.

Note: Messages acquired from different endpoints can be treated differently by the Integrated Endpoint process, depending upon the message conversion requirements of the endpoint (that is, bit-mapped ISO to fixed, field-separated to fixed, fixed to fixed) and how the message class is derived from the message.

This example assumes that all messages received from the endpoint are provided in bit-mapped ISO format and that the message class is derived from the product ID, message class, and message variant provided in the message.

Detailed processing is not depicted for the Integrated Authorization Server or Interface Master processes; only the processing specifically performed by the Integrated Endpoint process is depicted. Each of the processing steps are depicted in the illustration on the following page. A key to the abbreviations follows the illustration. Step descriptions are provided following the key.



Key

- IAS = Integrated Authorization Server process
- IEP = Integrated Endpoint process
- IMP = Interface Master process
- ITLF = ITS Transaction Log File

The following steps summarize transaction processing for a typical ITS application. In this example, a transaction request is acquired from an endpoint and is authorized by an Integrated Authorization Server process.

1. An endpoint acquires a transaction request and passes it to the XPNET process for processing.
2. The XPNET process determines the destination of the message using the DESTINATION attribute in the Network Environment File (NEF) for the endpoint. In this example, the destination is the symbolic name of an Integrated Endpoint process. The XPNET process delivers the message to the specified Integrated Endpoint process.
3. The Integrated Endpoint process receives the transaction request over the \$RECEIVE file and performs as follows:
 - a. The XPNET Interface object reads the source symbolic map extended memory table to obtain the source class for the endpoint station.
 - b. The XPNET Interface object formats an Object ID Retrieve member function and calls the Class object with the source class obtained from the source symbolic map extended memory table to determine the message handler object for the message.
 - c. The Class object reads the Source Class Map File extended memory table to obtain the appropriate message handler object for the message, which in this case, is an Endpoint Class Handler object. The Class object returns the object ID of the message handler object to the XPNET Interface object in an Object ID Retrieve member function response.
 - d. The XPNET Interface object formats a Message In member function, and forwards the application message to the endpoint-specific Endpoint Class Handler object.
 - e. The endpoint-specific Endpoint Class Handler object determines that the message is in native ISO format. It calls the Message ISO object with a Message Expand member function to expand the message from bit-mapped ISO format into fixed Internal Transaction Data (ITD) format.
 - f. The Message ISO object expands the native message. The expanded message is returned to the Endpoint Class Handler object.
 - g. The Endpoint Class Handler object determines the message class, message type, and message class wildcard flag values for the expanded message. The object sets up the appropriate defaults for the message to be routed, and builds the message class with the product ID set based on the products supported by the Endpoint Class Handler object and the contents of the message. The first four bytes of the message class and the message variant are filled in normally. A wildcard value is used for

any of the message class subfields, if indicated by the message class wildcard flag. The value of the message class wildcard flag is obtained from an application-specific file or extended memory table (for example, the Interface extended memory table).

Note: The actual fields used to build the message class can vary from one endpoint to another.

- h. The Endpoint Class Handler object determines that this particular message does not have stored context associated with it, thereby identifying the message as a new message. Therefore, a context key is not formatted.
- i. The Endpoint Class Handler objects sets the acquirer time in a Message Route member function and sends the function to the Endpoint Class Handler Services object to route the message.
- j. The Endpoint Class Handler Services object formats a Destination Retrieve member function to retrieve the destinations for the message. The Destination Retrieve member function is sent to the Destination object.
- k. The Destination object searches the Destination File extended memory table for the source class and message class provided to obtain the destinations for the message. In this case, the Destination object determines that the transaction is to be authorized by an Integrated Authorization Server process. The object formats a Destination Retrieve member function response and sends it back to the Endpoint Class Handler Services object.
- l. The Endpoint Class Handler Services object formats an External Message member function for the specified destination. This request is added to the front of the message area.
- m. The Endpoint Class Handler Services object formats a Message Out member function and sends it to the XPNET Interface object to deliver the message to the Integrated Authorization Server process for authorization.

The Message Out member function indicates that the destination expects to receive a Transport Layer Object to Transport Layer Object lexicon request in the message. The flag is set to true because the destination is an ITS process, where a transport layer object will receive the message.

- n. The XPNET Interface object formats a Message to Object member function in the Transport Layer Object to Transport Layer Object lexicon and places it at the front of the message text area.

- o. The XPNET Interface object formats a network message header to be delivered to the symbolic destination contained in the Message Out member function and delivers the message to the XPNET process.
- 4. The XPNET process receives the message and routes it to the symbolic destination listed in the message header.
- 5. The Integrated Authorization Server process receives the message over the \$RECEIVE file and calls the Interface Master process for station validation using a writeread interface.

The Interface Master process updates station, interface, and key counter information.

- 6. The Integrated Authorization Server process determines whether or not the transaction is to be authorized by the Integrated Authorization Server process itself or be routed to an external entity (for example, a host) for authorization. For the sake of this example, assume the transaction is authorized by the Integrated Authorization Server process itself.

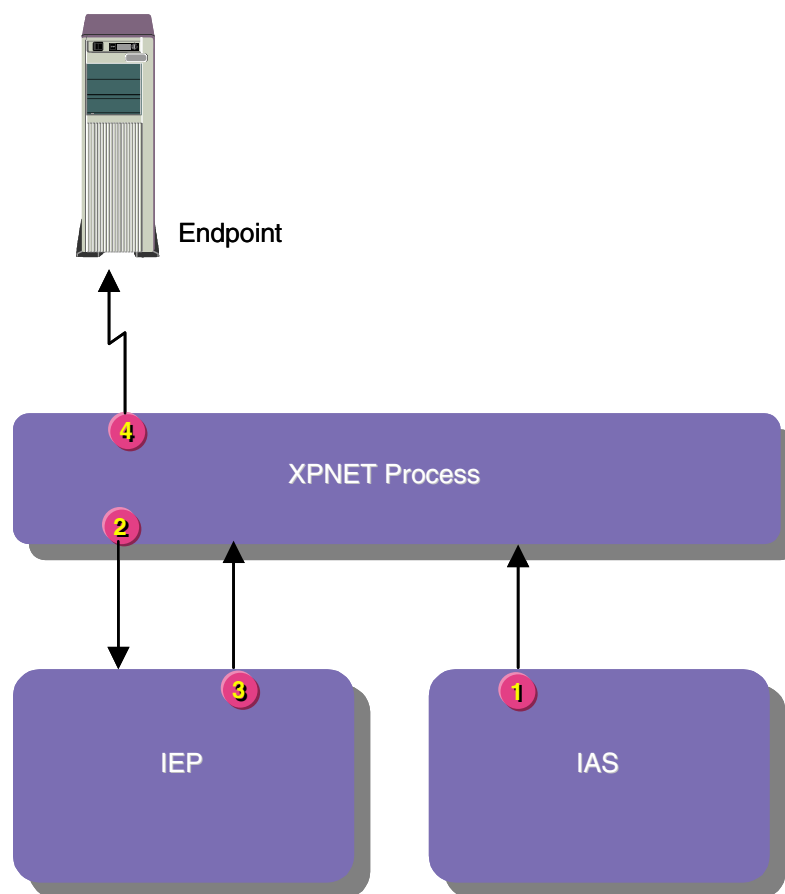
The Integrated Authorization Server process authorizes or denies the transaction, records the transaction in the ITS Transaction Log File, and creates a compressed (bit-mapped ISO format) transaction response.

Transaction Response Generated by an Integrated Authorization Server Process

The following transaction flow depicts the flow of information through the Integrated Endpoint process when a transaction response message is generated by an Integrated Authorization Server process.

This flow assumes that a transaction request message has already been processed and that the transaction response is routed back to the originating request endpoint. Message-based routing is not used for the response. Only the response portion of the transaction is depicted.

Detailed processing is not depicted for the Integrated Authorization Server process; only the processing specifically performed by the Integrated Endpoint process is depicted. Each of the processing steps are depicted in the illustration on the following page. A key to the abbreviations follows the illustration. Step descriptions are provided following the key.



Key

IAS = Integrated Authorization Server process

IEP = Integrated Endpoint process

The following steps summarize transaction processing for a typical ITS application. In this example, a transaction response from an Integrated Authorization Server process is routed back to the originating acquiring endpoint.

1. An Integrated Authorization Server process authorizes a transaction and generates a response message. In this case, a Message Context Store member function is included in the response. The response message is delivered to the XPNET process.
2. A transaction response message is delivered by the XPNET process to the XPNET Interface object in an Integrated Endpoint process over the \$RECEIVE file.

3. The Integrated Endpoint process receives the financial response transaction in compressed bit-mapped ISO format and performs as follows:
 - a. The XPNET Interface object receives the message and reads the source symbolic map extended memory table to obtain the source class of the Integrated Authorization Server process.
 - b. The XPNET Interface object formats an Object ID Retrieve member function and calls the Class object with the source class obtained from the source symbolic map extended memory table to determine the message handler object for the message.
 - c. The Class object reads the Source Class Map File extended memory table to obtain the appropriate message handler object for the message from the given source. In this case, the message handler is the XPNET Interface object itself. The Class object returns the object ID of the message handler object to the XPNET Interface object in an Object ID Retrieve member function response.
 - d. Because the XPNET Interface object is the message handler object, a Message to Object member function is present in the message received. The XPNET Interface object sends a Message In member function to the destination object specified in the Message to Object member function. In this case, the specified destination object is the Endpoint Class Handler Services object. The Message Context Store member function is enveloped within the Message In member function.
 - e. The Endpoint Class Handler Services object uses the context, the context key, and a timer value from the Message Context Store member function and prepares a Context Store member function. The Endpoint Class Handler Services object stores response destination information and passes it to the Context Manager object in the additional data area of the Context Store member function. The destination information is used in the event of a response indicating a failed message or timeout. The Endpoint Class Handler Services object sends the Context Store member function to the Context Manager object.

The Endpoint Class Handler Services object also updates Measure user counters for the internal authorization count and time and the overall (internal plus external) authorization count and time. These counts are stored in the additional data area of the message context.
 - f. The Context Manager object stores the context and context key in the Context extended memory table and starts a timer using the timer value provided by the acquirer object in the Integrated Authorization Server process. The Context Manager object formats and returns a Context Store member function response to the Endpoint Class Handler Services object.

- g. The Endpoint Class Handler Services object formats a Message Out member function in the Object to Transport Layer object lexicon. The destination for the message is the destination from the Message Context Store member function included with the Message In member function. The request is sent to the destination transport layer object ID from the Message Context Store member function, which, in this case, is the XPNET Interface object.
 - h. The XPNET Interface object formats the network message header to be delivered to the symbolic destination contained in the Message Out member function.

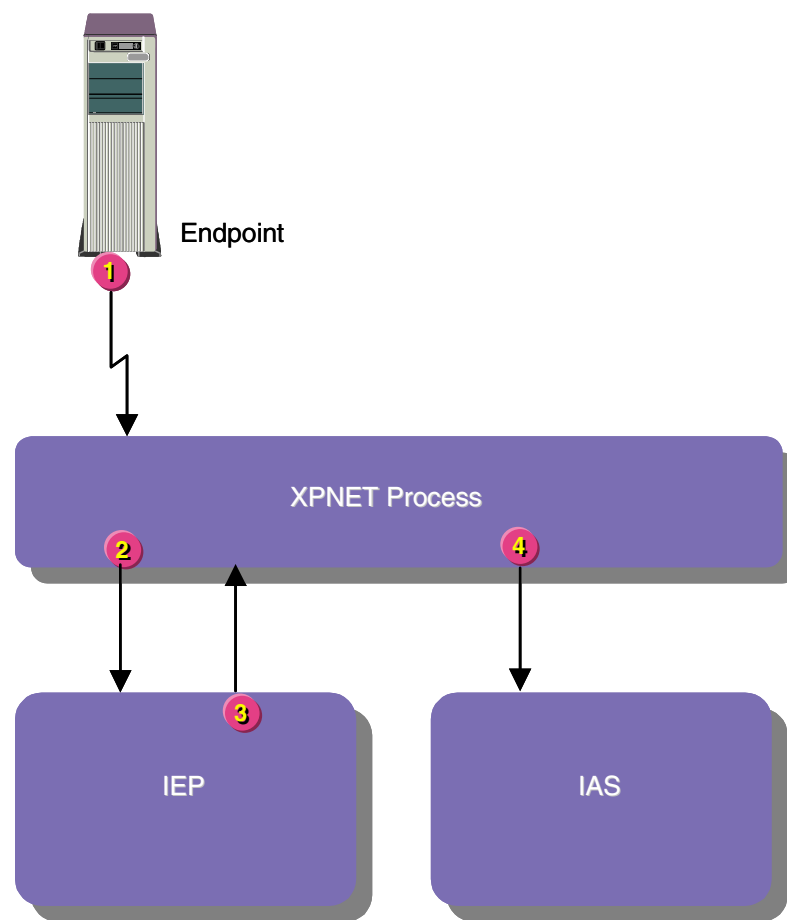
The lexicon flag in the Message Out member function indicates that the destination does not expect to receive a Transport Layer to Transport Layer lexicon request in the message, so none is added.
 - i. The message is delivered to the XPNET process.
- 4. The XPNET process delivers the transaction response to the originating endpoint.

Financial Completion Authorized by an Integrated Authorization Server Process

The following transaction flow depicts the flow of information through the Integrated Endpoint process when a financial completion message is received from an integrated endpoint and is authorized by an Integrated Authorization Server process.

This flow assumes that the financial request and response portions of a transaction have already been completed. Only the financial completion portion of the transaction is depicted.

Detailed processing is not depicted for the Integrated Authorization Server process; only the processing specifically performed by the Integrated Endpoint process is depicted. Each of the processing steps are depicted in the illustration on the following page. A key to the abbreviations follows the illustration. Step descriptions are provided following the key.



Key

IAS = Integrated Authorization Server process
IEP = Integrated Endpoint process

The following steps summarize transaction processing for a typical ITS application. In this example, a financial completion is received from an acquiring endpoint.

1. An endpoint passes a financial completion message to the XPNET process for processing.
2. The XPNET process determines the destination of the message using the DESTINATION attribute in the NEF for the endpoint. In this example, the destination is the symbolic name of an Integrated Endpoint process. The XPNET process delivers the message to the specified Integrated Endpoint process.

3. The Integrated Endpoint process receives the transaction request over the \$RECEIVE file and performs as follows:
 - a. The XPNET Interface object reads the source symbolic map extended memory table to obtain the source class for the endpoint station.
 - b. The XPNET Interface object formats an Object ID Retrieve member function and calls the Class object with the source class obtained from the source symbolic map extended memory table to determine the message handler object for the message.
 - c. The Class object reads the Source Class Map File extended memory table to obtain the appropriate message handler object for the message, which in this case, is an Endpoint Class Handler object. The Class object returns the object ID of the message handler object to the XPNET Interface object in an Object ID Retrieve member function response.
 - d. The XPNET Interface object formats a Message In member function, and forwards the application message to the endpoint-specific Endpoint Class Handler object.
 - e. The endpoint-specific Endpoint Class Handler object determines that the message is in native ISO format. It calls the Message ISO object with a Message Expand member function to expand the message from bit-mapped ISO format into fixed Internal Transaction Data (ITD) format.
 - f. The Message ISO object expands the native message. The expanded message is returned to the Endpoint Class Handler object.
 - g. The Endpoint Class Handler object receives the expanded transaction request message and determines the message class and message type for the message. Because the message type indicates a financial completion, which the Endpoint Class Handler object knows requires stored context, the Endpoint Class Handler object formats a context key. The Endpoint Class Handler object then formats a Message Route Context Retrieve member function and sends the request to the Endpoint Class Handler Services object.
 - h. The Endpoint Class Handler Services object formats a Context Retrieve member function in the Object to Context Manager lexicon. The request is sent to the Context Manager object.
 - i. The Context Manager object uses the context key provided in the request to retrieve the context. The object also deletes the timer associated with the context. The Context Manager object formats a Context Retrieve member function response and returns it to the Endpoint Class Handler Services object.

- j. The Endpoint Class Handler Services object accesses the additional data area returned by the Context Manager object to determine the destination of the completion message. The EPCH Lexicon flag indicates that the destination expects the Endpoint Class Handler Services Object to Object lexicon, so the Endpoint Class Handler Services object formats an External Message With Context member function and adds it to the front of the message.

The Endpoint Class Handler Services object then formats a Message Out member function in the Object to Transport Layer Object lexicon. The destination, destination object ID, lexicon flag, and the destination transport layer are all contained in the additional data area retrieved by the Context Manager object. The Message Out member function is sent to the XPNET Interface object.

- k. The Endpoint Class Handler Services objects sends a Destination Retrieve member function in the Object to Destination lexicon to the Destination object to determine whether any additional destinations need to process the completion. For the sake of this example, assume that no additional destinations are found.
- l. The XPNET Interface object receives the Message Out member function. The lexicon flag in the request indicates that the destination expects to receive a Transport Layer Object to Transport Layer Object lexicon request in the message. The XPNET Interface object formats a Message to Object member function in the Transport Layer Object to Transport Layer Object lexicon and places it at the front of the message text area.

The XPNET Interface object formats the network message header to be delivered to the symbolic destination contained in the Message Out member function.

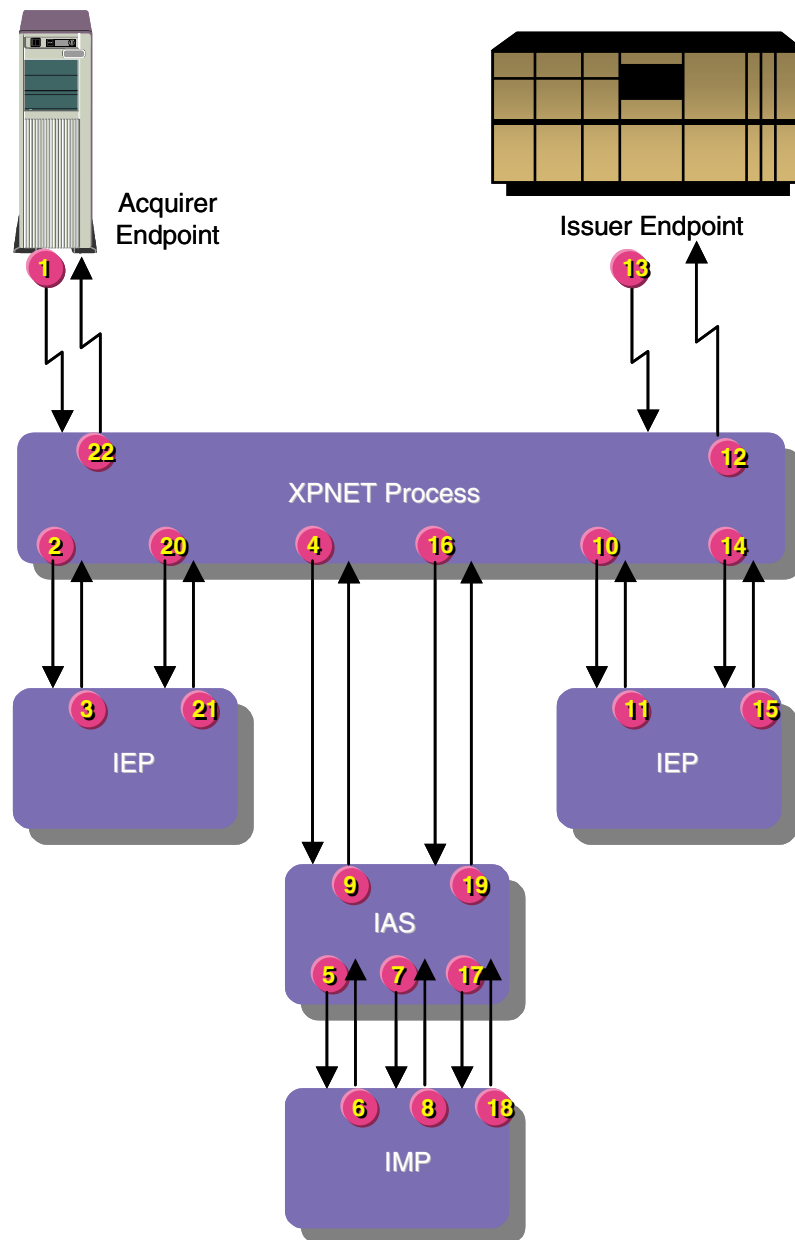
- m. The message is delivered to the XPNET process.
- 4. The XPNET process receives the message and routes it to the Integrated Authorization Server process.

Financial Transaction Authorized by an Endpoint

The following transaction flow depicts the flow of information through the Integrated Endpoint process when a financial transaction request is acquired from a terminal endpoint and is routed by an Integrated Authorization Server process to an external issuer endpoint for authorization. In this transaction flow two Integrated Endpoint processes are involved—one to handle the acquiring terminal endpoint and another to handle the issuer interface endpoint.

To avoid the unnecessary duplication of information, much of the detailed processing performed by the Integrated Endpoint processes has been left out, since it is similar to that described in previous transaction flows.

Each of the processing steps are depicted in the illustration on the following page. A key to the abbreviations follows the illustration. Step descriptions begin on the page following the illustration.



Key

IAS = Integrated Authorization Server process
 IEP = Integrated Endpoint process
 IMP = Interface Master process

The following steps summarize transaction processing for a typical ITS application. In this example, a financial transaction request is received from an acquiring endpoint and is routed by an Integrated Authorization Server process to an issuer endpoint for authorization.

1. A terminal endpoint passes a message to the XPNET process.
2. The XPNET process determines the destination of the message using the DESTINATION attribute in the NEF for the endpoint. In this example, the destination is the symbolic name of an acquirer Integrated Endpoint process. The XPNET process delivers the message to the acquirer Integrated Endpoint process.
3. The acquirer Integrated Endpoint process performs message-based routing and determines that the destination for the message to be an Integrated Authorization Server process. The Endpoint Class Handler object records the acquirer time and sends the transaction request to the XPNET process for delivery to the Integrated Authorization Server process.
4. The XPNET process delivers the transaction request to an Integrated Authorization Server process.
5. The Integrated Authorization Server process calls the Interface Master process to validate the endpoint station on which the transaction request was acquired.
6. The Interface Master process returns a response indicating whether the acquiring station is valid.
7. The Integrated Authorization Server process calls the Interface Master process to determine the issuer endpoint station to which the transaction request is to be sent.
8. The Interface Master process returns a response indicating the issuer endpoint station to which the transaction request is to be sent.
9. The Integrated Authorization Server process determines that the transaction request needs to be authorized by an issuer endpoint and routes the request to the XPNET process for delivery to the issuer Integrated Endpoint process.
10. The XPNET process delivers the transaction request to the issuer Integrated Endpoint process.

Context is stored and the Endpoint Class Handler Services object sets the external timestamp which is stored with the message context.
11. The issuer Integrated Endpoint process packages the transaction request and sends it to the XPNET process for delivery to the issuer endpoint.
12. The XPNET process delivers the transaction request to the issuer endpoint.

13. The issuer endpoint authorizes the transaction request, formats a transaction response, and sends it to the XPNET process for delivery to the issuer Integrated Endpoint process.
14. The XPNET process delivers the transaction response to the issuer Integrated Endpoint process.
15. The issuer Integrated Endpoint process retrieves context and updates the appropriate Measure user counters. The external authorization time is computed by subtracting the external timestamp from the current time. The external authorization count is increased by one. The transaction response is packaged and sent to the XPNET process for delivery to the Integrated Authorization Server process.
16. The XPNET process delivers the transaction response to the Integrated Authorization Server process.
17. The Integrated Authorization Server process sends a request to the Interface Master process to validate the issuer endpoint station on which the transaction response was authorized.
18. The Interface Master process returns a response indicating whether the issuing station is valid.
19. The Integrated Authorization Server process processes the response and sends it to the XPNET process for delivery to the acquiring Integrated Endpoint process.
20. The XPNET process delivers the response to the acquiring Integrated Endpoint process.
21. The Endpoint Class Handler Services object updates the appropriate Measure counters. The authorization count and the internal authorization count are increased by one. The authorization time is computed by subtracting the acquirer time set by the Endpoint Class Handler object from the current timestamp. The internal authorization time is computed by subtracting the external time from the authorization time. The transaction response is packaged and sent to the XPNET process for delivery to the originating acquirer endpoint.
22. The XPNET process delivers the response to the acquiring endpoint.

Context Timeout

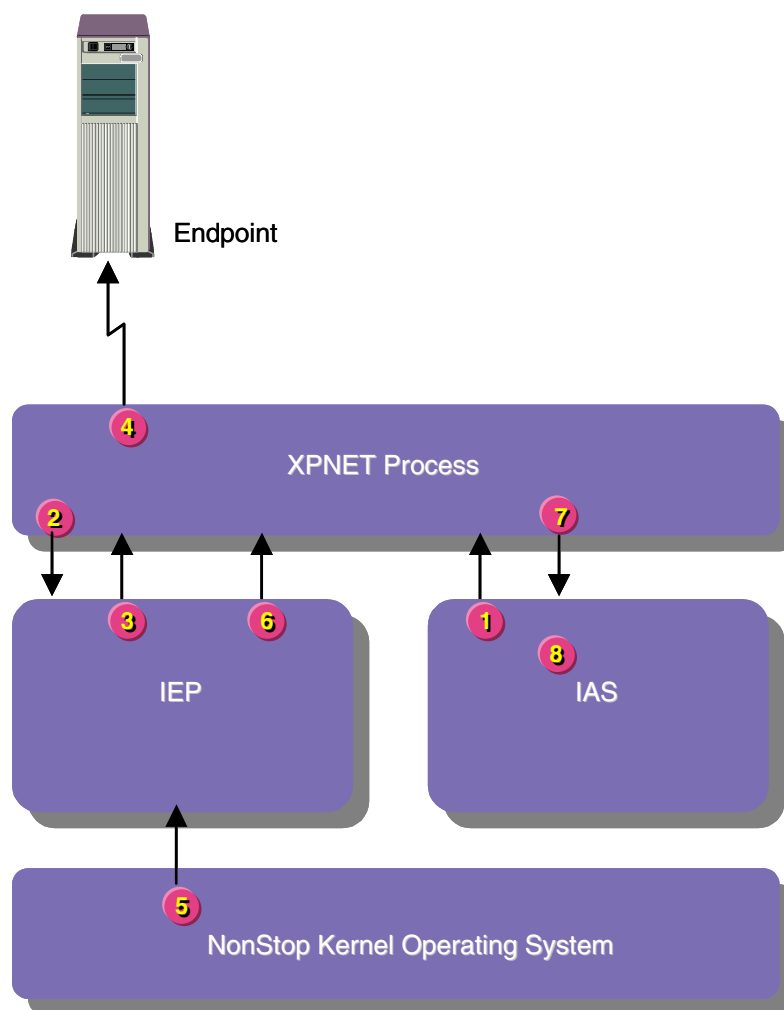
The following transaction flow depicts the flow of information through the Integrated Endpoint process when the timer for stored context from a transaction response expires before the associated financial completion is received and the context is retrieved.

In this flow, a financial response message is received from an Integrated Authorization Server process and is sent back to the originating transaction endpoint. Before the associated financial completion is received, the timer for the stored response context expires.

This example assumes that the associated transaction request and response have already been processed successfully.

Detailed processing is not depicted for the Integrated Authorization Server process; only the processing specifically performed by the Integrated Endpoint process is depicted.

Each of the context timeout processing steps are depicted in the illustration on the following page. A key to the abbreviations follows the illustration. Step descriptions are provided following the key.



Key

IAS = Integrated Authorization Server process

IEP = Integrated Endpoint process

The following steps summarize transaction processing when the context for a transaction response times out before the associated financial completion is received. In this example, transactions are acquired from an external endpoint and are authorized by an Integrated Authorization Server process.

1. An Integrated Authorization Server process authorizes a financial transaction, compresses the message, and sends it to the XPNET process. In this case, a Message Context Store member function is included in the response.
2. The XPNET process delivers the transaction response to the Integrated Endpoint process over the \$RECEIVE file.

3. The Integrated Endpoint process receives the financial transaction response transaction in compressed bit-mapped ISO format and performs as follows.
 - a. The XPNET Interface object reads the source symbolic map extended memory table to obtain the source class for the Integrated Authorization Server process.
 - b. The XPNET Interface object formats an Object ID Retrieve member function and calls the Class object with the source class obtained from the source symbolic map extended memory table to determine the message handler object for the message.
 - c. The Class object reads the Source Class Map File extended memory table to obtain the appropriate message handler object for the message, which, in this case, is the XPNET Interface object itself.
 - d. Because the XPNET Interface object is the message handler object, a Message to Object member function is present in the message received. The XPNET Interface object sends a Message In member function to the destination object specified in the Message to Object member function. In this case, the specified destination object is the Endpoint Class Handler Services object. The Message Context Store member function is enveloped within the Message In member function.
 - e. The Endpoint Class Handler Services object uses the context, the context key, and a timer value from the Message Context Store member function and prepares a Context Store member function. The Endpoint Class Handler Services object stores response destination information and passes it to the Context Manager object in the additional data area of the Context Store member function. The destination information is used in the event of a response indicating a failed message or timeout. The Endpoint Class Handler Services object sends the Context Store member function to the Context Manager object.

The Endpoint Class Handler Services object also updates Measure user counters for the internal authorization count and time and the overall (internal plus external) authorization count and time. These counts are stored in the additional data area of the message context.
 - f. The Context Manager object stores the context and context key in the Context extended memory table and starts a timer using the timer value provided by the acquirer object in the Integrated Authorization Server process. The Context Manager object formats and returns a Context Store member function response to the Endpoint Class Handler Services object.

- g. The Endpoint Class Handler Services object formats a Message Out member function in the Object to Transport Layer object lexicon. The destination for the message is the destination from the Message Context Store member function included with the Message In member function. The Message Out member function is sent to the destination transport layer object ID from the Message Context Store member function, which, in this case, is the XPNET Interface object.
 - h. The Endpoint Class Handler Services object sends a Destination Retrieve member function in the Object to Destination lexicon to the Destination object to determine whether any additional destinations need to process the response. For the sake of this example, assume that no additional destinations are found.
 - i. The XPNET Interface object formats the network message header to be delivered to the symbolic destination contained in the Message Out member function.

The lexicon flag in the Message Out member function indicates that the destination does not expect to receive a Transport Layer to Transport Layer lexicon request in the message, so none is added.
 - j. The message is delivered to the XPNET process.
- 4. The XPNET process delivers the transaction response to the originating endpoint.
 - 5. Before the associated financial completion can be generated by the acquiring endpoint and returned to the Integrated Endpoint process, the timer for the stored transaction response context in the Context extended memory table expires. The operating system informs the Integrated Endpoint process of a timer expiration.
 - 6. Upon receipt of the timer expiration notification, the Integrated Endpoint process performs as follows.
 - a. The XPNET Interface object formats a System Time Signal member function and forwards the request to the Context Manager object.
 - b. The Context Manager object retrieves the stored context and sends a Context Timeout member function to the object that originally requested that context be stored for the transaction. This is typically the Endpoint Class Handler Services object.
 - c. The Endpoint Class Handler Services object formats an External Message Timeout member function that includes the message context. The appropriate Measure user counters are also updated.
 - d. The Endpoint Class Handler Services object formats a Message Out member function and sends the request to the XPNET Interface object.

- e. The XPNET Interface object receives the Message Out member function. The lexicon flag in the Message Out member function indicates that the destination expects to receive a Transport Layer to Transport Layer lexicon request in the message. The XPNET Interface object formats a Message to Object member function in the Transport Layer to Transport Layer lexicon and places it at the front of the message text area.

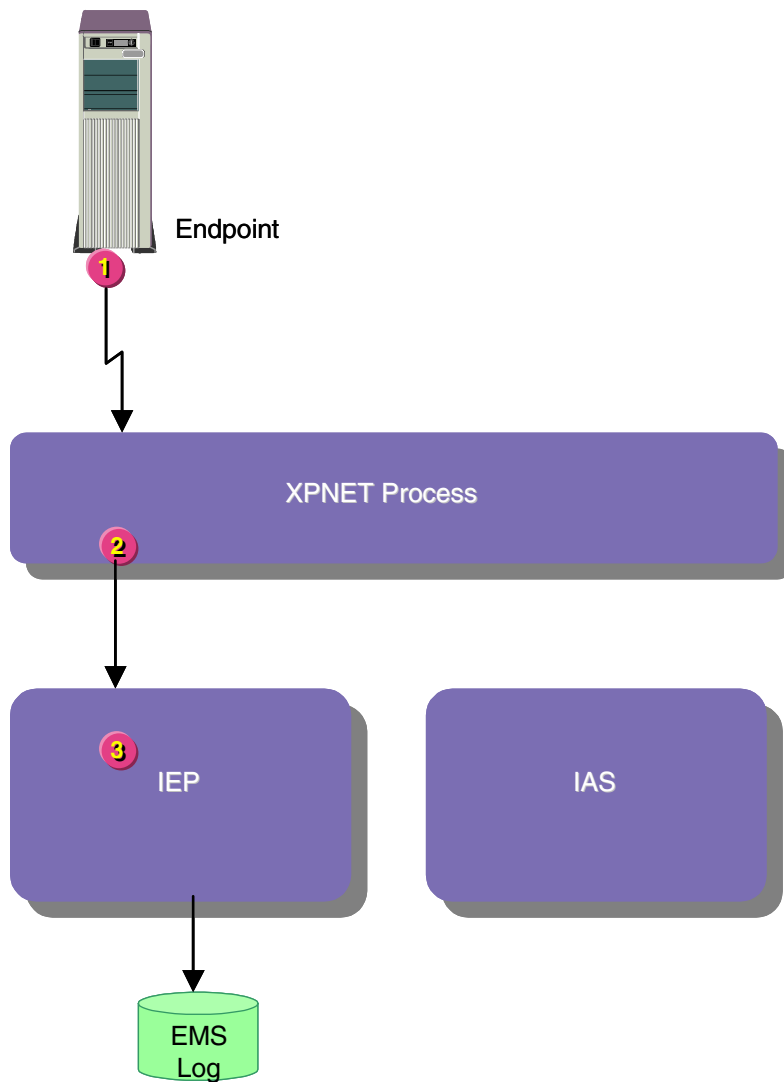
The XPNET Interface object formats the network message header to be delivered to the symbolic destination contained in the Message Out member function.

- f. The Message Out member function is delivered to the XPNET process.
- 7. The XPNET process delivers the message to the Integrated Authorization Server process that authorized the original transaction request.
 - 8. The Integrated Authorization Server process determines whether an advice is required.

Network Overload Detected

The following transaction flow depicts the flow of information through the Integrated Endpoint process when network overload is detected on a transaction request. Both the request and response legs of the transaction are depicted.

For clarity, only process-level interaction is shown in the transaction flow illustration on the following page. A key to the abbreviations follows the illustration. Step descriptions are provided following the key. Object interactions are not shown, although all object processing is provided in the processing step descriptions.

**Key**

EMS = Event Management Service
IAS = Integrated Authorization Server process
IEP = Integrated Endpoint process

The following steps summarize transaction processing when the network overload threshold for a transaction request is exceeded. In this example, transactions are acquired from an endpoint and are destined for an Integrated Authorization Server process.

1. The endpoint passes a financial request transaction message for an Integrated Endpoint process to the XPNET process.
2. The XPNET process determines the destination of the message using the DESTINATION attribute in the NEF for the endpoint. In this example, the destination is the symbolic name of an Integrated Endpoint process. The XPNET process delivers the message to the specified Integrated Endpoint process.
3. The Integrated Endpoint process receives the transaction request over the \$RECEIVE file and performs as follows:
 - a. The XPNET Interface object reads the source symbolic map extended memory table to obtain the source class for the station.
 - b. The XPNET Interface object formats an Object ID Retrieve member function and calls the Class object with the source class obtained from the source symbolic map extended memory table to determine the message handler object for the message.
 - c. The XPNET Interface object checks the value of the Network Overload Management Percent field in the network message header against the value of the NOM-PERCENT-THRESHOLD param in the Logical Network Configuration File (LCONF) to determine whether network overload is detected. For the sake of this example, assume that the value of the LCONF param has been exceeded. The XPNET Interface object changes the value of the Network Overload Management Percent field network message header to 101 to indicate that overload has been detected.
 - d. The Class object reads the Source Class Map File extended memory table to obtain the appropriate message handler object for the message, which in this case, is an Endpoint Class Handler object. The Class object returns the object ID of the message handler object to the XPNET Interface object in an Object ID Retrieve member function response.
 - e. The XPNET Interface object formats a Message In member function, and forwards the application message to the endpoint-specific Endpoint Class Handler object.

- f. The endpoint-specific Endpoint Class Handler object determines that the message is in native ISO format. It calls the Message ISO object with a Message Expand member function to expand the message from bit-mapped ISO format into fixed Internal Transaction Data (ITD) format.
- g. The Message ISO object expands the native message. The expanded message is returned to the Endpoint Class Handler object.
- h. The Endpoint Class Handler object determines the message class, message type, and message class wildcard flag values for the expanded message. The object sets up the appropriate defaults for the message to be routed, and builds the message class with the product ID set based on the products supported by the Endpoint Class Handler object and the contents of the message. The first four bytes of the message class and the message variant are filled in normally. A wildcard value is used for any of the message class subfields, if indicated by the message class wildcard flag. The value of the message class wildcard flag is obtained from an application-specific file or extended memory table (for example, the Interface extended memory table).

Note: The actual fields used to build the message class can vary from one endpoint to another.

- i. If context does not exist for the message and the message is a transaction request, the Endpoint Class Handler object sends the Message Header Get Destination Queue State member function to retrieve the value of the Network Overload Management Percent field from the network message header. For the sake of this example, assume that the value returned is 101 indicating an overload situation. In this case, the message is discarded. This is the automatic network overload function of the Integrated Endpoint process.
- j. No further processing of the transaction request is performed. A response with a message discarded status is formatted by the Endpoint Class Handler object in the Message In member function response.

The XPNET Interface object updates a discard counter and starts a timer based on the value of the QED LCONF param. After this timer expires, it outputs an appropriate error message to the Event Management Service (EMS) log.

ACI Worldwide Inc.

Glossary

This glossary explains terms commonly associated with the ITS Integrated Endpoint process environment.

Broadcast routing — A method of sending messages to multiple destinations in parallel. Broadcast routing allows a message to be sent to an unlimited number of destinations at the same time. This allows for parallel processing of a particular message by multiple applications.

Chained routing — A method of sending messages serially through a number of endpoints. Each endpoint does some work with the message and sends it back to an Integrated Endpoint process for further routing to another endpoint. At a minimum, either the message class or the source class must differ from one endpoint to the next.

Context — Any information stored over the course of a transaction that is used to maintain the context of the transaction. Context typically consists of a stored copy of a message and a timer associated with the message. When the next message (for example, an acknowledgement) for a transaction is received, the context is retrieved and the timer is deleted. If the transaction times out before the next message is received, the Integrated Endpoint process uses the stored context to recreate the original message and issue an appropriate error message.

Endpoint — An entity through which messages are sent or received, such as a device, interchange, host, or process. An endpoint is physically represented by a process or station on the network. An endpoint is logically represented by an endpoint class. Examples of endpoints include the following: an ATM device, a POS device, an Interchange, a host, a remote banking device, and an Integrated Authorization Server process.

Endpoint class (EPC) — A set of transaction sources or destinations that share a common message format. In general, an endpoint class is defined by a set of message formats. Thus, an endpoint class is defined for each interface, device, and process type residing in the system, since they all have unique message formats.

Even if two endpoint classes have similar message formats, they are likely to be considered different because the way the Endpoint Class Handler object communicates with them differs.

Endpoint Class Handler object — An Endpoint Class Handler is an object that contains all of the functions and procedures required to process the message types for a particular endpoint class. Whereas the endpoint class is a set of transaction types, the Endpoint Class Handler object is a set of procedures for processing the transaction types defined by the endpoint class. The Endpoint Class Handler object can interpret the format of messages received from the class. The Endpoint Class Handler object is capable of selecting the appropriate fields from a message to derive the message class. Any time a new endpoint class is added with new message formats (for example, a new device or interface), a new Endpoint Class Handler object must be created.

Integrated Endpoint process — A satellite process to the XPNET process responsible for message-based routing, symbolic destination routing, maintaining transaction context, Measure count manipulation, and network overload management. The Integrated Endpoint process maintains transaction context for certain transaction phases. It also provides message-based and symbolic destination routing capabilities to any application connected to the XPNET process. An Integrated Endpoint process always contains an Endpoint Class Handler object.

Lexicon — A group of messages used to provide communication between objects. For objects to converse with each other they must share at least one of the same lexicons.

Message class — A set of message types. In general, a message class groups similar messages. An example of a message class is network management messages. Another example could be ATM withdrawal transactions. The message class for a message is determined by the Endpoint Class Handler object that handles messages from the endpoint using the message content. Message-based routing of messages is based on the source class and message class of a message.

Object — A combination of data and procedural statements that act upon that data. An object is called by sending it a message. The object interprets the message and performs the appropriate action. Objects typically encompass specific processing functions allowing for smaller, more easily manageable units of code that can accommodate high transaction volumes.

Object ID — An identification number for an object. The object IDs provided by ACI should not be changed.

Object-oriented design (OOD) — A software development methodology utilizing small, reusable, easily managed units of code called objects. Objects are used as building blocks to create processes.

Object type — A means of categorizing objects for configuration and control purposes. When an object is called, it must be of the correct object type. The object type is used mainly for application objects (that is, core objects and endpoint-specific Endpoint Class Handler objects); kernel objects typically do not have an object type.

Pass-through routing — A method of routing of messages received from an endpoint station to another station, without entering an application. Messages are simply passed through the Tandem system.

Simple routing — The routing of messages for a particular source class and message class received from an endpoint to an online processing destination.

Source class — A means of grouping similar endpoints as one logical entity for message routing. The source class is defined for each endpoint in the NEFS using the Sourceclass element for systems running under XPNET 2.1 or greater or the Class element for systems running under XPNET 2.0.

Transport layer object (TLO) — An object used to communicate with the transport layer. Currently, XPNET is the only transport layer supported. The XPNET Interface object allows ITS processes to communicate with the XPNET process.

ACI Worldwide Inc.

Index

B

Broadcast routing, definition of, [Glossary-1](#)
Building processes, [1-12](#)

C

Chained routing, definition of, [Glossary-1](#)
Context Manager object
 configuration of, [2-8](#)
 context extended memory table, [2-5](#)
 context retrieval, [2-5](#)
 context storage, [2-5](#)
 introduction, [2-1](#)
 member functions received, [2-9](#)
 member functions sent, [2-9](#)
 normal processing, [2-5](#)
 overview, [2-2](#)
 STATUS command, [2-7](#)
 timeout processing, [2-7](#)
Context Retrieve member function, [1-27](#)
Context Store member function, [1-27](#)
Context Timeout member function, [1-19](#)
Context, definition of, [Glossary-1](#)
Core objects, [1-12](#)

D

Destination File (DEST) documentation, [1-31](#)
Destination object
 configuration of, [3-11](#)
 destination retrieval, [3-4](#)
 initialization, [3-4](#)
 introduction, [3-1](#)
 member functions received, [3-12](#)
 member functions sent, [3-12](#)
 overview, [3-2](#)
 processing overview, [3-4](#)
 search key, [3-5](#)
 search processing, [3-9](#)
 wildcard routing, [3-4](#)
Destination Retrieve member function, [1-28](#)

E

Endpoint Class Handler object
 configuration of, [4-13](#)
 definition of, [Glossary-2](#)

device-specific functions, [4-9](#)
EMT notify member function processing, [4-11](#)
exception handling, [4-11](#)
extended memory table usage, [4-11](#)
generic Endpoint Class Handler object, [4-12](#)
initialization, [4-6](#)
introduction, [4-1](#)
member functions received, [4-15](#)
member functions sent, [4-14](#)
message conversion, [4-7](#)
message failure processing, [4-9](#)
message reception, [4-7](#)
message routing, [4-8](#)
normal processing, [4-7](#)
optional processing, [4-11](#)
overview, [4-3](#)

Endpoint Class Handler Services object
 configuration of, [5-13](#)
 destination retrieval, [5-5](#)
 introduction, [5-1](#)
 measure counters, [5-7](#)
 member functions received, [5-15](#)
 member functions sent, [5-14](#)
 message routing, [5-4](#)
 normal processing, [5-4](#)
 overview, [5-2](#)
 timer management, [5-12](#)

Endpoint class, definition of, [Glossary-1](#)

Endpoint, definition of, [Glossary-1](#)

External Message Failed member function, [1-22](#)

External Message Failed with Context member function, [1-23](#)

External Message Late member function, [1-23](#)

External Message member function, [1-22](#)

External Message Timeout member function, [1-23](#)

External Message with Context member function, [1-22](#)

F

Field-separated message processing, [4-8](#)

Fixed structure message processing, [4-7](#)

G

Generic Endpoint Class Handler object, [4-12](#)

I

Integrated endpoint objects

- Context Manager, [2-1](#)
- Destination, [3-1](#)
- Endpoint Class Handler, [4-1](#)
- Endpoint Class Handler Services, [5-1](#)
- introduction, [1-11](#)
- Message ISO, [6-1](#)

Integrated Endpoint process

- assumptions, [1-3](#)
- definition of, [Glossary-2](#)
- description of, [1-2](#)
- transaction processing flows, [7-1](#)

Interface extended memory table, [4-11](#)International Organization for Standardization (ISO), [4-8](#)

ISO message components

- data element, [6-5](#)
- header, [6-4](#)
- message type, [6-4](#)
- primary bit map, [6-4](#)
- secondary bit map, [6-5](#)

ISO-format message processing, [4-8](#)**K**Kernel objects, [1-11](#)**L**Lexicon, definition of, [Glossary-2](#)

Logical Network Configuration File (LCONF) assigns

- CTXEMT-SWAPVOL, [2-8](#)
- DESTEMT, [3-11](#)

Logical Network Configuration File (LCONF) params

- CTXEMT-CONTEXT-PAGES, [2-8](#)
- QED, [4-10](#)

M

MAC

see Message authentication code (MAC)

MAC Generate/PIN Translate member function, [1-25](#)MAC Verify member function, [1-25](#)Measure position counters, [5-10](#)

Member functions

- Context Retrieve, [1-27](#)
- Context Store, [1-27](#)
- Context Timeout, [1-19](#)
- Destination Retrieve, [1-28](#)
- External Message, [1-22](#)
- External Message Failed, [1-22](#)
- External Message Failed with Context, [1-23](#)
- External Message Late, [1-23](#)
- External Message Timeout, [1-23](#)
- External Message with Context, [1-22](#)
- introduction, [1-17](#)

MAC Generate/PIN Translate, [1-25](#)MAC Verify, [1-25](#)Message Compress, [1-30](#)Message Compress Unique, [1-25](#)Message Context Store, [1-29](#)Message Expand, [1-30](#)Message Expand Unique, [1-25](#)Message Route, [1-20](#)Message Route Context Retrieve, [1-20](#)Message Route Symbolic, [1-20](#)Message Trace Unique, [1-25](#)Object ID Retrieve, [1-28](#)Register, [1-20](#), [1-27](#)Version, [1-18](#)Message authentication code (MAC), [1-25](#)

Message class

- definition of, [Glossary-2](#)
- usage, [5-8](#)

Message class wildcard flag, [3-5](#), [5-5](#)Message Compress member function, [1-30](#)Message Compress Unique member function, [1-25](#)Message Context Store member function, [1-29](#)Message Expand member function, [1-30](#)Message Expand Unique member function, [1-25](#)

Message ISO object

- configuration of, [6-7](#)
- introduction, [6-1](#)
- ISO external message format, [6-4](#)
- member functions received, [6-9](#)
- member functions sent, [6-8](#)
- message compression, [6-5](#)
- message expansion, [6-5](#)
- normal processing, [6-4](#)
- overview, [6-2](#)

Message member function, [1-29](#)Message Route Context Retrieve member function, [1-20](#)Message Route member function, [1-20](#)Message Route Symbolic member function, [1-20](#)Message Trace Unique member function, [1-25](#)Message type, [5-9](#)Message variant, [5-9](#)**N**Nucleus process terminology change, [xiv](#)**O**Object ID Retrieve member function, [1-28](#)Object ID, definition of, [Glossary-3](#)Object type, definition of, [Glossary-3](#)Object, definition of, [Glossary-2](#)Object-oriented design, definition of, [Glossary-3](#)

P

Pass-through routing, definition of, [Glossary-3](#)
Procedural software compatibility, [1-4](#)

R

Register member function, [1-20](#), [1-27](#)

S

Simple routing
 definition of, [Glossary-3](#)
 example of, [7-2](#)
Source class, definition of, [Glossary-3](#)
Symbolic destination routing, [1-7](#)

T

Transaction flows
 completion, [7-11](#)
 context timeout, [7-19](#)
 interface authorization, [7-15](#)
 network overload detected, [7-24](#)
 online issuer endpoint authorization, [7-15](#)
 request, [7-2](#)
 response, [7-7](#)
Transport layer object, definition of, [Glossary-3](#)

V

Version member function, [1-18](#)

X

XPNET process definition, [xiv](#)

ACI Worldwide Inc.